GENERAL LLM RESEARCH · PLANNING DOCUMENT V1.0

A Unified System for LLM Training, Compression, Inference & Deployment

Plan, roadmap, requirements and goals for testing, analysing and crafting optimal dense, Mixture-of-Experts and scaled multi-agent systems — derived from seven LLM-optimisation research papers.

7
source papers synthesised (quantisation, pruning, KV/attention, distillation & lossless, NVFP4, learned sparse attention, speculative decoding)

11
phased work packages with 5 go/no-go gates over ≈40 weeks

10+
custom kernel families specified, parameterised by a hardware descriptor

9
UI mockups across web and desktop shells

Date: 30 October 2026 · **Status:** Draft for review · **Audience:** project owner and the implementing coding agent (Claude Code)
Scope of this document: architecture, requirements, design rationale, data/storage process plans, UI mockups, kernel layout specifications, roadmap and ordered checklist. It contains *pseudocode and schemas only*; no production code.
Working codename: "Forge" (placeholder — rename freely).

HOW TO READ THE NUMBERS IN THIS DOCUMENT

Figures are tagged by origin. P1–P7 mean "reported in source paper N" (the papers themselves label many of their numbers as author-reported, recalled-from-memory, vendor-claimed or simulated). DERIVED means computed here from stated formulas. TARGET means a goal we set, to be recalibrated after the Phase-1 baselines. UI mockups use **illustrative placeholder data** and are not measurements.

Contents

1 Executive Summary 4
1.1 What we are building and why 4
1.2 Twelve design principles distilled from the papers 4
1.3 Headline decisions of this plan 5
1.4 Honest limits of the source material 6
2 Goals, Scope and Success Metrics 7
2.1 Goals 7
2.2 Scope 8
2.3 Success metrics and initial targets 8
2.4 Three model forms, three problem shapes 9
3 Requirements 10
3.1 Hardware abstraction and profiling 10
3.2 Kernels 10
3.3 Compression 11
3.4 Training and distillation 12
3.5 Inference and deployment 12
3.6 Evaluation 13
3.7 Data, interface and non-functional requirements 13
4 System Architecture 15
4.1 Layered architecture 15
4.2 Layer responsibilities and the reasoning behind each 15
4.3 Control-plane API (resource model) 16
4.4 Technology choices and alternatives 17
5 Recipe Planner and Cost Model 18
5.1 The cost model: bytes first 18
5.2 Rule base: the papers' decision procedures as data 20
5.3 Search loop (pseudocode) 21
5.4 Calibrating the cost model 21
5.5 The claims ledger 22
6 Hardware Abstraction and Custom Kernel Layouts 23
6.1 Core idea: spec → layout → schedule 23
6.2 What the chip descriptor drives 23
6.3 Layout derivation algorithm (pseudocode) 24
6.4 Kernel catalogue 25
6.5 Chip-family mapping 28
6.6 Autotuning and measurement protocol 29
6.7 Fusion map and numerical edge cases 30
7 Model Forms, Training and Deployment Plans 31
7.1 Dense models 31
7.2 Mixture-of-Experts models 32
7.3 Scaled multi-agent fleets 32
7.4 Training and distillation system 34
7.5 Inference and deployment 35
8 Data, Weights and Log Storage: Process Plans 37
8.1 Principles 37
8.2 Model weights: master + derived variants + deltas 38
8.3 Checkpoints: chunk dedup, delta chains, retention 39
8.4 Training data: shards, manifests, dedup 39
8.5 Teacher and student data 39
8.6 Logs, metrics and evaluation outputs 40
8.7 Retention, promotion and garbage collection 41
8.8 Access paths and performance 41
9 Interfaces: Web App and Desktop App 42
9.1 Options compared 42
9.2 What the desktop shell adds 42
9.3 Screen inventory 43
9.4 Prototype mockups 44

10 Evaluation, Validation and Risk **49**
10.1 Evaluation protocol and gates 49
10.2 Verification of the platform itself 50
10.3 Risk register 50
11 Roadmap **52**
11.1 Phases 52
11.2 Gates 54
11.3 Resources and assumptions 54
11.4 Working with Claude Code: handoff conventions 54
12 Ordered Checklist: What to Do, in What Order **56**
Appendix A — Traceability: Findings → Requirements → Work **65**

Source papers and short labels

Label	Paper (as attached)	Primary subject
P1	Quantization of Large Language Models — weights, activations, KV cache, training-time low-precision formats	PTQ/QAT, rotations, vector quantisation, FP8/MX/NVFP4 formats, Marlin-style kernels, decision guide
P2	Pruning and Sparsity in Large Language Models	Unstructured / 2:4 / structured / depth / expert pruning, activation sparsity, prune+distill pipelines
P3	KV-Cache, Attention, and Context Compression	GQA/MLA/hybrids, eviction, trainable sparse attention, paging, prefix caching, disaggregation, prompt compression
P4	Distillation, Low-Rank and Tensor Factorization, Parameter Sharing, Lossless Weight Compression	On-policy distillation, whitening-SVD, delta storage (BitDelta), entropy-coded weights (DFloat11, ZipNN)
P5	NVFP4 and Microscaled 4-Bit Floating-Point Formats for LLM Training and Inference	E2M1 grid, two-level scaling, stochastic rounding, RHT, training recipes, block-scaled GEMM, failure modes
P6	Learned Sparse and Hybrid Sparse–Linear Attention for Long-Context LLM Inference	DSA/NSA/MoBA, shared indexers, byte accounting, hybrids, eviction/quantisation interplay, evidence ledger
P7	Next-Generation Speculative Decoding — EAGLE-3, P-EAGLE, Speculative Speculative Decoding	Exactness, roofline regime analysis, trees, parallel drafting, SSD, engine integration

1 Executive Summary

1.1 What we are building and why

The seven papers describe a field that has split into many techniques — quantisation, pruning, distillation, low-rank factorisation, lossless coding, KV-cache reduction, learned sparse attention and speculative decoding — each with its own tooling, its own evaluation habits and its own hardware assumptions. Their shared message is that **no single technique is "the" answer**: the right configuration depends on which resource binds (capacity, bandwidth, compute or accuracy margin), on the chip, on the workload, and on whether training budget is available (P1 §19.1).

This document therefore plans one coherent system — "Forge" — whose job is: **given a model (or model family), a target chip or cluster, a workload profile and an accuracy budget, test and analyse the candidate configurations, and produce the best validated training, compression and serving recipe — together with the kernels, data plumbing and interfaces needed to run it** — for three model forms: **dense**, **Mixture-of-Experts (MoE)**, and **scaled multi-agent fleets** (many cooperating model instances sharing weights, caches and context).

> **THE SYSTEM IN ONE PARAGRAPH**
> A hardware profiler measures the chip; a roofline-based *recipe planner* proposes configurations using the decision procedures of P1–P4 and P6–P7; a *kernel library*, generated from a hardware descriptor and autotuned, executes them; a *compression and training engine* produces artifacts; an *evaluation harness* (built first, because perplexity is not an acceptable acceptance test) gates every artifact; a *content-addressed artifact fabric* stores everything compactly with full lineage; and a web app and desktop app expose all of it through one control-plane API.

1.2 Twelve design principles distilled from the papers

#	Principle	Evidence in the papers	Consequence for Forge
1	**Bytes are the currency of decode.**	Decode sits far left of the roofline ridge, so latency ∝ bytes moved; 16→4-bit weights buys ≈3–4× if the kernel dequantises at full bandwidth (P1 F1; P3 F1; P7 abstract).	Every plan is costed by a byte-accounting / roofline model *before* any experiment is launched.
2	**The binding resource picks the technique.**	Capacity → any weight compression; small-batch bandwidth → weight-only; large batch → W/A formats; long context → KV work (P1 §19.1).	The recipe planner starts with "which resource binds?", not "which algorithm?".
3	**Perplexity is necessary, never sufficient.**	Perplexity hides losses in long-context, reasoning, safety and calibration (P1 F9; P2 F7: +3.5% perplexity vs +83.7% on a stereotype-reliance score; P3 F7).	The evaluation harness is built in Phase 0, and no artifact can be promoted without passing KL / flip-rate / long-context / safety gates.
4	**Formats and kernels are hardware-specific.**	Recipes are increasingly format-specific, raising lock-in risk (P1 abstract); NVFP4 needs TN-layout, padded + swizzled scale tensors (P5 §11.2).	Kernels and weight layouts are parameterised by a *chip descriptor*; artifacts carry a `layout_id`.

5	**Fuse the quantiser.**	Unfused FP4 activation quantisation costs 59% of GEMM time at N=4096, so 3× peak becomes ≈2.3× (P5 Table 11.2, derived).	Quantise-in-prologue / epilogue is a kernel requirement, not an optimisation.
6	**Structured pruning + distillation is the most reliable route to a smaller dense model.**	Robustly ranked first under every weighting (P2 §1.3); Minitron, Nemotron Elastic (P2 F3).	The compression engine treats prune→distill as a first-class pipeline, with unstructured pruning as a research tool only.
7	**A compression method must beat "train or distil a smaller model".**	Over-trained small models and densing-law trends keep moving the baseline (P4 F8).	Every experiment automatically includes a "smaller-model-of-equal-footprint" baseline arm.
8	**Speculation is a regime-dependent tool.**	EAGLE-3: up to 6.5× at batch 1 but 1.38× at batch 64 (SGLang) and 1.01× at batch 56 (vLLM) (P7); adaptive γ that includes 0 is the safe default.	The serving engine owns a load-aware speculation controller; the simulator shows the break-even batch.
9	**For sparse attention, the selector's own scan can dominate.**	Index scan = 88% of bytes at 128K in a V3.2-style layer; sharing across 4 layers lifts byte reduction 7.7× → 22.6× (P6).	Indexer kernels are low-precision and shareable across layers; the byte-accounting tool reports selector bytes explicitly.
10	**Lossless layers are the safest wins.**	Paging, prefix caching, chunked prefill, disaggregation, tiered offload (P3 F5); entropy-coded BF16 ≈ 30% smaller, bit-identical (P4 F7).	These are on by default in serving and in cold storage; lossy steps are opt-in and gated.
11	**Architecture does most of the KV work; post-hoc eviction fails on hard tasks.**	Qwen3-Next ≈ 24 KiB/token and Kimi Linear ≈ 8 KiB/token vs 144 KiB/token for dense GQA Qwen3-8B (P3 derived); eviction weak on multi-hop / long CoT (P3 F4).	The planner recommends model-architecture choices when training is possible, and conservative KV tools when it is not.
12	**Most evidence is author-reported or vendor-reported.**	P5 had no Blackwell access; P7 ran no real-GPU experiments; P6 flags vendor claims; P1/P2 mark numbers "approximate / recalled".	A *claims ledger* is a product feature: each claim the planner relies on is linked to a reproduction experiment and a status.

1.3 Headline decisions of this plan

- **Evaluation before optimisation.** Phase 0 builds the harness (KL divergence, flip rate, long-context, reasoning, safety, noise-aware statistics). P5's noise audit showed most 12B benchmark differences are statistically indistinguishable from zero; Forge bakes confidence intervals into every comparison.
- **Production-default recipes first, frontier second.** Following the P1 priority tiers: W4A16 PTQ, FP8 inference and Marlin-class kernels first (Tier 1); block-scaled FP4, QAT/QAD, KV quantisation next (Tier 2); vector/trellis sub-3-bit, FP4 *training* and ternary last (Tier 3, gated).
- **Hardware abstraction via descriptors, not forks.** One kernel specification language, instantiated per chip family, with autotuning and an explicit portability matrix (§6).
- **One compact, content-addressed store.** Weights are stored once as a master plus *derived* variants and deltas; checkpoints are chunk-deduplicated; teacher knowledge is stored as top-k logits rather than full vocabularies (≈ 680× smaller, derived in §8); cold weights are entropy-coded losslessly.
- **Two interface shells, one API.** A web app (team, remote clusters) and a desktop app (local GPUs, offline, native telemetry) share a front-end code base and a control-plane API; both are specified with mockups in §9.

- **Multi-agent is an explicit hypothesis track.** The papers support its building blocks (prefix caching, delta serving, context folding, speculative n-gram lookup) but do not evaluate agent fleets end to end; Phase 10 is therefore gated and measured, not assumed.

1.4 Honest limits of the source material

TREAT THE PAPERS AS A MAP, NOT AS GROUND TRUTH

The seven documents are literature syntheses written with AI assistance, and say so. Their priority rankings are explicitly “subjective judgements”. Several headline speed-ups are vendor numbers (P5, P6); P7 reports that in a production engine verification dominates and measured speed-ups fall well short of theoretical bounds as batch size grows; P6 calls frontier-scale sparse-attention parity “thinly evidenced” beyond 106 tokens. Forge’s Phase 1 therefore **re-measures** the key claims on hardware we actually have access to, and the recipe planner’s defaults are updated from those measurements rather than from the papers.

2 Goals, Scope and Success Metrics

2.1 Goals

ID	Goal	Why it matters / what "done" looks like
G1	**Measure before believing**	Reproduce the load-bearing claims of P1–P7 on available hardware; maintain a claims ledger with status *reproduced / contradicted / untested*. Done = every default in the recipe planner points at a ledger entry.
G2	**Find the best configuration per (model, chip, workload, accuracy budget)**	Automate the decision procedures of P1 §19, P2 §1.2, P3 Ch. 16, P4 §1.3, P7 §19.3 and validate their output by experiment. Done = planner top-1 is within 5% of the exhaustive-search optimum on a held-out set of (model, chip) pairs.
G3	**Hardware-optimal kernels for any chip**	A kernel specification and autotuning flow that instantiates efficient GEMM / attention / MoE / utility kernels from a chip descriptor, with a portability fallback for unknown chips. Done = ≥ 85% of achievable memory bandwidth on bandwidth-bound kernels on each supported chip (TARGET).
G4	**Support dense, MoE and multi-agent forms**	Each form has its own planner rules, kernels, stores and UI views (§7). Done = one reference model of each form flows through the full pipeline.
G5	**Train and distil under low precision safely**	BF16 → FP8 → (gated) NVFP4 precision ladder, prune→distil pipelines, on-policy distillation, QAT/QAD. Done = reference student meets accuracy gate at ≤ 25% of parent footprint (TARGET).
G6	**Serve efficiently and predictably**	Paged KV, prefix caching, chunked prefill, optional disaggregation, sparse/latent attention, load-aware speculation. Done = a published capacity curve (tokens/s vs batch × context) per deployment target.
G7	**Store everything compactly with full lineage**	Content-addressed, deduplicated, tiered store; derived-artifact model; entropy-coded cold weights; top-k teacher caches. Done = measured compaction ≥ 3× vs a naive "one directory per run" layout (TARGET).
G8	**Make the whole thing observable and usable**	Web and desktop apps with comparison, Pareto, kernel and storage views. Done = any two runs can be compared in ≤ 3 clicks; a new user reaches a first baseline report in < 1 hour.

2.2 Scope

IN SCOPE

- Decoder-only transformers (dense, MoE, GQA/MLA, hybrid linear/attention).
- Post-training quantisation, QAT/QAD, FP8 and (gated) FP4 training.
- Structured width/depth/expert pruning with distillation; activation sparsity.
- Low-rank/whitened-SVD, delta storage, lossless entropy coding.
- KV-cache reduction (architecture, quantisation, paging, offload, sparse attention).
- Speculative decoding (EAGLE-style, n-gram, parallel drafting; SSD as an experiment).
- Kernel specification, autotuning, correctness and performance harness.
- Artifact/data fabric, experiment tracking, web + desktop apps.

OUT OF SCOPE (V1)

- Training frontier-scale models from scratch (we target pruning/distillation/continued training and small-scale pre-training studies).
- Non-transformer-only architectures as primary targets (SSM/linear layers are supported only inside hybrids).
- Vision/audio/video generation models (formats may transfer, but evaluation does not).
- Proprietary-model extraction: teachers are limited to models whose terms permit distillation (see risk R9).
- Building a general-purpose cluster scheduler (we integrate with existing ones).
- Writing the production code in this document — that is for Claude Code, using §12 and the appendices as the brief.

2.3 Success metrics and initial targets

All targets are TARGETs chosen to be ambitious but consistent with what the papers report as achievable; they are recalibrated at Gate G1 after the baseline phase measures what our hardware actually does.

Area	Metric	Initial target	Reference in papers
Kernel efficiency	Achieved / peak HBM bandwidth, W4A16 decode GEMM, B ≤ 16	≥ 85%; speed-up vs BF16 GEMM ≥ 3.2× (ideal 4×)	P1 §14: Marlin stays near-max 4× to B=16–32
Kernel efficiency	Fused block-scaled FP4 GEMM vs FP8 GEMM, compute-bound shapes	≥ 2.2× on 2×-ratio parts; ≥ 2.6× on 3×-ratio parts (hardware permitting)	P5 Table 11.2 (unfused ≈2.3× at N=4096)
End-to-end	Decode tokens/s vs BF16 baseline, 8B and 70B reference models, B=1	W4A16 ≥ 2.5×; with spec-decode ≥ 3.5× (dense, short ctx)	P1 F1, P7 (6.5× best case at B=1)
Accuracy	Suite score retained vs parent (see §10 gates)	FP8 ≥ 99.5%; W4 ≥ 98% (≥ 30B), ≥ 96% (< 10B); FP4 ≥ 97% (≥ 30B)	P1 F2; P5 §19.1 (97–99% for ≥ 30B NVFP4)
Accuracy	Top-1 token flip rate vs parent, greedy, 1k prompts	FP8 ≤ 1%; W4 ≤ 4% (gate, not goal)	P1 §19 validation step
Long context	RULER / HELMET / NoLiMa-style score retention at target length	≥ 95% of parent at 128K for KV-reduction recipes	P3 F7; P6 §13.2

Memory	KV bytes per token at target length (dense 8B GQA: 128 KiB)	≥ 4× reduction via quantisation + paging; ≥ 10× via latent/sparse recipes where training is allowed	P3 F1–F3
Storage	Bytes on disk for a 100-run experiment tree vs naive	≥ 3× smaller; cold weights ≥ 25% smaller than BF16 with bit-exact restore	P4 F7 (≈30% for BF16)
Planner	Agreement of planner top-1 with exhaustive-search optimum	≥ 80% exact, ≥ 95% within 5% of optimum throughput at equal accuracy	—
Reproducibility	Re-running a deterministic stage yields identical artifact hash	100% for deterministic stages; stochastic stages within pre-registered noise band	P5 noise audit
Usability	Clicks to compare two runs; time to first baseline report	≤ 3 clicks; < 1 hour on a supported machine	—

2.4 Three model forms, three problem shapes

The same toolbox is applied differently depending on form. This table is the one-page justification for why the plan treats them as separate tracks.

Form	What binds	Highest-value levers (with source)	Distinct risks
Dense	Weight bytes at small batch; KV bytes at long context; compute at large batch	W4A16/FP8 (P1); structured prune+distil (P2); GQA/MLA/hybrid architecture (P3); learned sparse attention (P6); spec decode at low batch (P7); TEAL-style activation sparsity 1.3–1.8× (P2)	Depth-pruned models lose generative ability (P2 F4); low-rank rarely wins wall-clock (P4 F5)
MoE	Total-parameter memory (all experts resident) and all-to-all communication; decode touches few experts so bandwidth per token is lower but capacity is huge	Router-aware expert pruning/merging, 25–50% experts with small loss (REAP; P2 F5); MoE-aware quantisation (P1, Tier 3); grouped GEMM; quantised all-to-all; expert offload tiers (P3)	Spec decode can *invert* its batch dependence (P7 §5.8); calibration must cover rarely-used experts; MoE quantisation is "young" (P1)
Scaled multi-agent	Aggregate KV/context across many sessions; repeated prefixes; many model variants; orchestration latency	Prefix/radix caching (P3 top tier); delta/adapter serving, BitDelta >10× per fine-tune (P4); context folding & typed eviction policies (P3 F6); n-gram speculation for context-copying agents (P7 §19.3); disaggregated prefill/decode (P3)	Papers do not evaluate fleets end to end; hybrid/linear layers showed multi-hop deficits at scale (P3 F7); compression may degrade tool-use reliability (to be measured)

3 Requirements

Requirements are numbered for traceability (Appendix A maps each paper finding → requirement → work package → checklist item). Priority follows MoSCoW: MUST blocks release, SHOULD is planned but may slip, COULD is an option. "Basis" cites the paper whose finding motivates the requirement.

3.1 Hardware abstraction and profiling

ID	Requirement	Pri	Basis
HW-1	A **chip descriptor** (schema in Appendix B) captures: compute units, per-unit shared memory / register file, tensor-core dtypes and tile shapes, supported low-precision formats (FP8, MXFP4/6/8, NVFP4, INT8/INT4), peak FLOP/s per dtype, HBM capacity and bandwidth, L2 size, interconnect topology and bandwidth.	MUST	P1, P5
HW-2	A **capability probe** auto-fills the descriptor from the driver/runtime, and a **micro-benchmark suite** measures achievable (not datasheet) bandwidth, MMA throughput per dtype, shared-memory bandwidth, launch latency and dequantisation ALU throughput. Datasheet values are never used for decisions when a measurement exists.	MUST	P1 §14.1, P5 §11, P7 §5
HW-3	The effective ridge point (FLOP/byte) is stored per dtype and per kernel class; P7 notes real stacks have a *lower effective ridge* than the datasheet.	MUST	P7 §5.8
HW-4	Support tiers: Tier A = NVIDIA Hopper- and Blackwell-class (FP8; FP4/NVFP4 on Blackwell); Tier B = AMD Instinct-class (FP8; FP4/FP6 on newer parts), Apple-silicon/CPU via portable kernels; Tier C = generic fallback (portable, unoptimised). Vendors' low-precision features are only enabled after the probe confirms them.	SHOULD	P1 F6
HW-5	Multi-device topology awareness (NVLink/PCIe/Infinity-Fabric class links, NIC bandwidth) for tensor-, expert- and pipeline-parallel planning and for disaggregated prefill/decode.	SHOULD	P3 F5, P7
HW-6	Power/energy sampling where exposed, so energy per token is a reportable metric.	COULD	P1 §2.2.4

3.2 Kernels

ID	Requirement	Pri	Basis
KR-1	Every kernel is defined by a **kernel spec** (op, dtypes, tile shape ranges, pipeline depth, layout_id, epilogue fusions) plus a **reference implementation** in plain tensor code used as the correctness oracle.	MUST	P1 §14
KR-2	Mixed-precision weight-only GEMM (W4A16, W8A16, optionally W3/W6 bit-plane layouts) with offline weight reordering, multi-stage async copy, fast dequantisation, and per-group scales applied in the epilogue when mathematically equivalent.	MUST	P1 §14.2
KR-3	FP8 GEMM (per-tensor and block scaling) and block-scaled FP4 GEMM (NVFP4 and MXFP4 variants) with **fused activation quantisation**, padded + swizzled scale tensors, and TN-layout handling.	MUST	P5 §11.2–11.3
KR-4	Paged decode attention with GQA grouping (g query heads share a KV head to raise arithmetic intensity), split-KV parallelism for long context, FP8/INT4 KV dequantisation in-kernel, and an MLA (latent) variant.	MUST	P3 F1, P3 F2, P1 F8
KR-5	Sparse-attention kernels: low-precision lightning-indexer scan, top-k selection, block- or token-granular gather attention, and **cross-layer indexer sharing**.	SHOULD	P6 §12.7

KR-6	Speculative-decoding kernels: tree-attention with custom masks/position ids, fused rejection-sampling verification, KV accept/rollback compaction.	SHOULD	P7 §6, §12.1
KR-7	MoE kernels: fused routing/top-k, token permutation, grouped GEMM over variable expert batch sizes, quantised all-to-all, expert-parallel overlap.	SHOULD	P2 F5, P1 Ch.15
KR-8	Utility kernels: fast Walsh–Hadamard (block 16/32/128), absmax/scale computation, stochastic-rounding quantiser, TEAL-style thresholded gather-GEMV, and a **fused entropy-decode + GEMM prologue** for lossless weights.	COULD	P5 §6, P2 F5, P4 F7
KR-9	An **autotuner** searches tile shapes / pipeline depth / split-K over the spec's ranges per (chip fingerprint, shape bucket, dtype config), caches winners, and refuses to cache any candidate that fails the correctness oracle.	MUST	P1 §14.1
KR-10	Each kernel reports roofline position: achieved bytes/s and FLOP/s, % of measured peak, bound type, and (for sparse attention) selector bytes vs attention bytes.	MUST	P6 Ch.3

3.3 Compression

ID	Requirement	Pri	Basis
CP-1	PTQ engine with calibration pipeline: RTN baseline, GPTQ-family (Hessian $H = X^TX$ layer-wise reconstruction), activation-aware scaling (AWQ-style), SmoothQuant-style smoothing; group sizes 32–128; symmetric/asymmetric; calibration-set registry (calibration sensitivity is documented).	MUST	P1 F2–F4
CP-2	Format-aware quantisation for block-scaled FP4 (MR-GPTQ-style) with per-format scale search ("Four-Over-Six"-style choose-scale-by-error is an option).	SHOULD	P1 F6, P5 §4.9
CP-3	Rotation module (randomised Hadamard / learned rotations) applied only where the planner's per-format model says it helps; P5 finds RHT does *not* reduce RTN error for NVFP4 but does reduce SR-product variance 6.6–22×.	SHOULD	P1 F4, P5 §6
CP-4	KV-cache quantisation: per-channel keys / per-token values (KIVI-style), FP8 KV default, 4-bit KV behind a long-context gate.	MUST	P1 F8
CP-5	Structured pruning (width incl. GQA group structure, depth, attention heads, experts) driven by activation/Fisher-type importance, always paired with a distillation recovery stage; unstructured / 2:4 only as research options with explicit "no GPU speed-up expected" warnings.	MUST	P2 F1–F4
CP-6	MoE expert pruning/merging using router statistics, calibrated on a domain mix that includes generative samples.	SHOULD	P2 F5
CP-7	Training-free activation sparsity (TEAL-style magnitude thresholds) composable with weight quantisation.	COULD	P2 F5
CP-8	Low-rank: activation-whitened SVD (SVD-LLM-style) with heterogeneous rank allocation and a short healing stage; the planner blocks rank choices above the break-even rank $r < mn/(m+n)$ unless memory (not latency) is the goal.	SHOULD	P4 F4–F5
CP-9	Delta compression for fine-tune families (low-rank deltas, 1-bit-sign + scale à la BitDelta) with multi-tenant serving support.	SHOULD	P4 §1.3
CP-10	Lossless entropy coding of BF16/FP8 weights for storage and distribution (exponent-field coding, byte-grouped zstd as baseline); on-the-fly decode inside the GEMM only where a fused kernel exists.	SHOULD	P4 F7
CP-11	Mixed-precision bit allocation (per-layer/per-expert bit-widths) from sensitivity analysis under a byte budget.	COULD	P1 Ch.9

CP-12	Every recipe is a **declarative pipeline** (YAML, Appendix C) whose stages are individually cacheable and whose output artifact hash is a pure function of inputs + recipe.	MUST	—

3.4 Training and distillation

ID	Requirement	Pri	Basis
TR-1	Precision ladder per training run: BF16 reference → FP8 mixed precision (production-proven) → NVFP4 (gated, experimental). Each rung is a recipe with per-layer high-precision exemptions and a final high-precision "healing" phase option.	MUST	P1 F7, P5 §8
TR-2	FP4-training diagnostics: zero-gradient-fraction per 16×16 weight tile, scale-underflow fraction, E4M3 scale-range occupancy, SR bias test, loss-gap vs reference run. (P5 reproduces a failure where low-norm channels produce zero weight gradients and total MSE hides it.)	MUST	P5 §15.4
TR-3	Distillation modes: offline logit (top-k cached), sequence-level, **on-policy** (student samples, teacher scores), reasoning-trace SFT; temperature, forward/reverse KL, JSD configurable.	MUST	P4 F1–F2
TR-4	Prune→distil pipeline (Minitron-style): importance estimation, structured prune, distil with ≥ several-billion-token budget (planner warns below), nested-model extraction as an option.	MUST	P2 F3, §1.2
TR-5	QAT and quantisation-aware *distillation* (QAD) from a BF16 teacher; QAD is preferred for RL-trained reasoning models where task-loss QAT lost most accuracy.	SHOULD	P5 Ch.10, P1 F5
TR-6	Continued-training adaptation to sparse/latent attention (e.g., DSA-style indexer training with KL objective; MLA conversion) with a baseline arm for the same token budget.	SHOULD	P3 F3, P6 Ch.8
TR-7	Draft-model training for speculative decoding (EAGLE-3-style "training-time test", multi-layer feature fusion) and acceptance-per-position reports.	SHOULD	P7 Ch.8
TR-8	Teacher-use policy check (licence / terms of service) is a blocking gate for any distillation job whose teacher is not locally owned or permissively licensed.	MUST	P4 F3

3.5 Inference and deployment

ID	Requirement	Pri	Basis
SV-1	Serving engine (or engine adapter layer for an existing open engine) with continuous batching, paged KV, radix prefix caching, chunked prefill; each switchable per deployment and measured.	MUST	P3 F5
SV-2	Optional disaggregated prefill/decode and tiered KV offload (HBM → host DRAM → NVMe → remote) with block-hash addressing shared with the artifact fabric.	SHOULD	P3 F5
SV-3	**Load-aware speculation controller**: per-request γ chosen from measured acceptance rates and current batch; γ=0 allowed; reports tokens/step, acceptance per position and verification cost. Exactness (output distribution unchanged) is a tested invariant.	MUST	P7 §3, §5.8
SV-4	Engine bookkeeping for speculation: ragged per-sequence progress, KV rollback, draft-state management, CUDA-graph capture of draft loops, tree masks; interactions with grammar-constrained decoding, prefix caching, chunked prefill, quantised KV and multi-LoRA tested.	SHOULD	P7 §12.1
SV-5	Capacity planner: given chip, model artifact, context distribution and SLOs → max concurrency, tokens/s, cost per million tokens, with the byte-accounting breakdown (weights / KV / selector / activations).	MUST	P1 §19.3, P6 Ch.3

SV-6	Deployment targets: single GPU, multi-GPU node, multi-node, workstation/desktop, and edge/CPU (GGUF-class export). Release process: shadow → canary → ramp, with automatic rollback on eval-gate regression.	SHOULD	P1 Ch.14
SV-7	Multi-agent fleet runtime: shared base weights + adapters/deltas, cross-agent prefix sharing, per-agent budgets and context-folding policies, fleet-level scheduler and trace capture.	SHOULD	P3 F6, P4, P7 §19.3

3.6 Evaluation

ID	Requirement	Pri	Basis
EV-1	Distribution-level metrics vs the parent: KL divergence on held-out text, top-1 flip rate, entropy shift, calibration (ECE).	MUST	P1 §19
EV-2	Task suites: knowledge, code, math, instruction following, multilingual; **free-form long generation** (errors accumulate over thousands of CoT tokens).	MUST	P1 F9, P2 F7
EV-3	Long-context suite beyond needle-in-a-haystack: RULER, HELMET, NoLiMa, BABILong, SCBench-style multi-turn reuse; run at the *deployment* context length.	MUST	P3 F7, P6
EV-4	Safety, refusal and bias probes paired with every compression change (pruning and quantisation can shift them independently of perplexity).	MUST	P2 F7, P1 F9
EV-5	Statistics: every comparison reports bootstrap/binomial confidence intervals and a "distinguishable from noise?" flag; seeds and sample counts pre-registered per experiment.	MUST	P5 Ch.12
EV-6	Agentic evaluation: multi-turn tool-use and multi-hop tasks, because hybrid/sparse/compressed models showed deficits there that average benchmarks missed.	SHOULD	P3 F7
EV-7	All generations stored (compressed) so any later metric can be recomputed without re-running models (see §8).	SHOULD	—

3.7 Data, interface and non-functional requirements

ID	Requirement	Pri
DT-1	Content-addressed, chunk-deduplicated artifact store with manifests, signed lineage DAG, three tiers and mark-and-sweep GC (§8).	MUST
DT-2	Weights stored as *master + derived variants + deltas*; each derived artifact records its recipe hash and can be regenerated on demand.	MUST
DT-3	Training data as versioned, tokenised, packed shards with mixture manifests, dedup report and per-shard statistics; teacher outputs cached as top-k log-probabilities keyed by (teacher hash, prompt hash, sampling params).	MUST
DT-4	Logs and metrics in a columnar telemetry lake with rollups and retention; raw event logs retained short, aggregates long.	MUST
UI-1	Web app and desktop app share one front-end code base and one control-plane API; the desktop shell adds local GPU telemetry, offline mode and native file handling.	MUST
UI-2	Views: command centre, hardware/roofline, recipe builder, experiment comparison with Pareto frontier, kernel lab, training monitor, serving simulator, storage explorer, fleet topology (§9).	MUST
UI-3	Every number shown carries provenance (measured here / reproduced / paper-reported / derived) and a confidence interval where applicable.	SHOULD
NFR-R1	**Reproducibility:** deterministic stages are bit-reproducible given hashes; stochastic stages record seeds and are compared within pre-registered bands.	MUST

NFR-P1	**Performance of the platform itself:** artifact store sustains ≥ 70% of device sequential bandwidth for checkpoint save/load; UI interactions < 200 ms on cached data.	SHOULD
NFR-S1	**Security:** signed manifests, least-privilege job credentials, no secrets in logs, licence/terms metadata on every model and dataset, audit trail of who ran which teacher.	MUST
NFR-V1	**Portability:** every accelerated kernel has a reference fallback; unknown chips degrade to Tier C instead of failing.	MUST
NFR-O1	**Observability:** traces from request → scheduler → kernel; cost and energy accounting per run.	SHOULD
NFR-E1	**Extensibility:** new methods, formats and chips are added as plug-ins (recipe stage, kernel spec, chip descriptor) without touching the core.	MUST

4 System Architecture

4.1 Layered architecture

Forge is organised as seven layers with a strict rule: **a layer only calls the layer directly below it**. This keeps hardware knowledge (L2–L3) out of the planner and UI, and keeps experiment bookkeeping (L1) out of the kernels. The same stack is used for all three model forms; what changes is the content of L4 (model forms) and the recipes that L6 proposes.

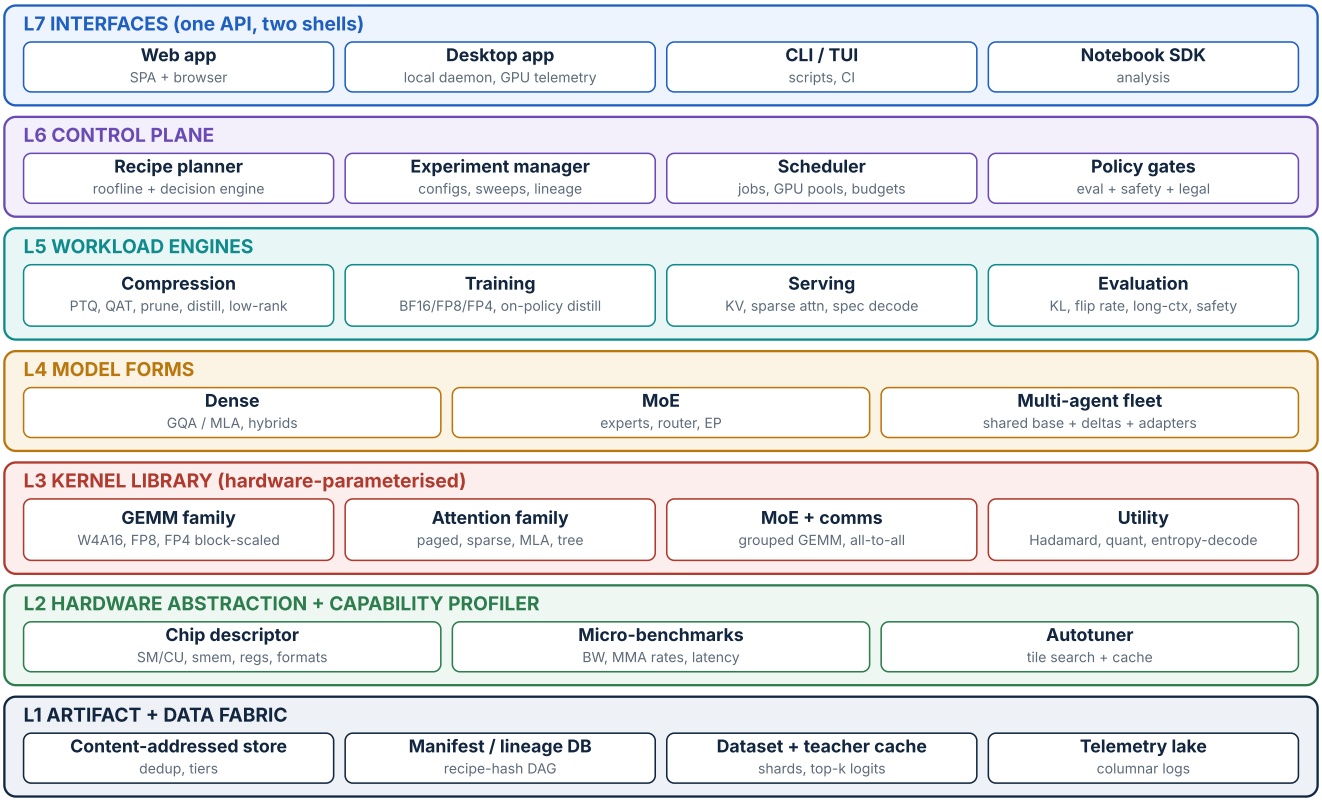


Figure 4.1 Layered architecture. L1 (artifact + data fabric) and L2 (hardware abstraction) are the two foundations; everything above is configuration and orchestration over them.

4.2 Layer responsibilities and the reasoning behind each

Layer	Responsibility	Why it is a separate layer
L1 Artifact + data fabric	Content-addressed store, manifests, lineage DB, dataset and teacher caches, telemetry lake (§8).	Compactness and reproducibility are properties of storage, so they must not depend on which tool wrote the bytes. Isolating L1 lets the same store back training, compression, evaluation and serving.
L2 Hardware abstraction + profiler	Chip descriptors, capability probes, micro-benchmarks, autotuner and its cache (§6).	P1 and P5 show best recipes are format- and chip-specific; isolating measurement behind one interface lets the planner reason about “a chip” as data.

L3 Kernel library	GEMM, attention, MoE, utility kernels generated from specs; reference implementations; correctness oracles (§6).	Kernel work is slow, specialised and the main source of performance variance; quarantining it behind kernel specs lets us swap implementations (hand-written, template-generated, vendor library) without touching recipes.
L4 Model forms	Dense, MoE and multi-agent fleet descriptions: tensor inventories, parallelism plans, cache structures (§7).	The forms differ in what binds (§2.4), so each owns its own cost-model terms and planner rules.
L5 Workload engines	Compression, training, serving and evaluation engines, each executing declarative pipeline stages.	Stages are cacheable units: same inputs + same recipe → same artifact hash.
L6 Control plane	Recipe planner (§5), experiment manager, scheduler, policy gates (evaluation, safety, licence).	Decision logic is the product's differentiator and must be inspectable and testable apart from execution.
L7 Interfaces	Web app, desktop app, CLI/TUI, notebook SDK (§9).	All interfaces speak one API so features appear everywhere simultaneously and automation equals UI.

4.3 Control-plane API (resource model)

The API is resource-oriented. The same objects appear in the UI, the CLI and the SDK. Only the shape is specified here; Claude Code owns the concrete implementation.

Resource	Key fields	Notes
`Chip`	descriptor, probe results, microbench results, fingerprint	Immutable once measured; re-measure creates a new version.
`Model`	form (dense/moe/fleet), architecture config, tensor inventory, licence, parent hash	Architecture config is parsed into the byte-accounting model.
`Workload`	context-length distribution, batch/concurrency profile, SLOs (TTFT, ITL, tokens/s), prefix-reuse rate, task mix	Can be imported from traces; synthetic generators for what-if.
`Recipe`	declarative stage list (Appendix C), parameters, expected cost	Hashable; diffable; templated from the planner.
`Run`	recipe hash, inputs, status, metrics stream, outputs	One run = one recipe execution on one pool.
`Artifact`	content hash, kind (weights, calib stats, logits cache, eval gens…), derived-from edges, recipe hash	The lineage DAG of Figure 8.2.
`EvalReport`	suite versions, per-metric value + CI, noise flags, comparison to parent, gate verdict	Gate verdict is machine-readable and blocks promotion.
`Deployment`	artifact set, topology, engine config, rollout state	Links back to the capacity plan that justified it.
`Claim`	source paper + location, statement, status (reproduced/contradicted/untested), linked experiments	Powers the claims ledger (§5.5).

4.4 Technology choices and alternatives

Concern	Recommendation	Rationale / alternative considered
Control plane & engines	Python 3.12+, typed (pydantic-style schemas), async job runner.	Ecosystem for PyTorch-class tooling; schemas double as the API contract. Alternative: Rust control plane — rejected for v1 (slower iteration), kept for the store daemon.
Artifact store & daemon	Rust (or Go) service with memory-mapped chunk store; BLAKE3 hashing; zstd.	Throughput and safety for multi-GB streaming; also used as the desktop app's local daemon.
Kernels	Layered: reference in framework tensor ops → portable Triton-class kernels → vendor-tuned CUTLASS/CuTe-class (NVIDIA) and HIP/CK-class (AMD) variants for hot ops.	Portable kernels give coverage on unknown chips; tuned variants give the last 15–30% where it matters. All share one kernel spec.
Metadata DB	SQLite (desktop/single user) and PostgreSQL (team), same schema via migrations.	One schema, two scales.
Telemetry lake	Parquet files + DuckDB-class embedded query engine; optional object-store backing.	Columnar compression is excellent for metrics; embedded query keeps desktop offline-capable.
Web front-end	TypeScript SPA (React-class) with a charting layer capable of 100k+ points (canvas/WebGL), served by the control plane.	Large time series and heat maps need canvas rendering.
Desktop shell	A lightweight webview shell (Tauri-class) embedding the same SPA plus a native Rust sidecar for GPU telemetry and file access; Electron-class is the fallback.	Shares ≈90% of UI code; smaller footprint than a full browser runtime; native access where required (§9.2).
Job execution	Adapters for local process pool, SSH hosts, and common cluster schedulers.	We integrate rather than build a scheduler (out of scope).

5 Recipe Planner and Cost Model

The planner is the part of Forge that turns the papers' prose ("W4A16 if small-batch decode is bandwidth-bound; FP8 W8A8 first if native FP8 cores exist; …") into executable, testable logic. It has three parts: a **cost model** (physics), a **rule base** (the papers' decision procedures) and a **search loop** (experiments that test and refine both).

5.1 The cost model: bytes first

All formulas below are standard and appear in P1 (§2.2, §14.1, §19), P3 (Ch. 2) and P6 (Ch. 3); the numerical examples are DERIVED here.

Quantity	Formula	Used for
Weight bytes	$W = P \cdot b_{\text{eff}} / 8$, where b_{eff} includes scales (e.g., NVFP4 4.5, MXFP4 4.25, W4 g128 ≈ 4.1)	Fit test; decode bytes
KV bytes per token	$k = 2 \cdot L_{\text{cache}} \cdot H_{\text{kv}} \cdot d_{\text{h}} \cdot b_{\text{kv}} / 8$ (MLA: latent width instead; hybrids: only full-attention layers count)	Concurrency; context limit
Decode step time (ideal)	$t \approx (W_{\text{active}} + B \cdot T \cdot k \cdot \varphi$ + selector scan) / β_{eff}, with φ the fraction of cache read	Latency/throughput bound
Arithmetic intensity	GEMM ≈ $2B$ / (b/8) FLOP per weight byte; decode attention ≈ g FLOP/byte for a group of g query heads per KV head	Bound type; ridge crossover
Batch crossover (W4A16)	$B^* = \Pi_{\text{tc}} \cdot (b/8) / (2 \cdot \beta)$ (P1: ≈ 74 vs ≈ 295 for FP16 on H100-class numbers)	Where weight-only stops helping
Quantiser overhead	$T_q/T_g = (b_{\text{in}}+b_{\text{out}}) \cdot F / (2N \cdot \beta)$ (P5 eq. 11.1)	When fusion is mandatory
Speculation gain	E[tokens/step] and cost ratio c → speedup $S(\alpha, \gamma, c)$; break-even near $B \approx \text{E}[N] \cdot n_{\text{eff}}^* / (\gamma+1)$ (P7 §5.8)	γ controller defaults
Amdahl	End-to-end speed-up = $1 / (\Sigma_i\, f_i/s_i)$ over weight GEMMs, attention, KV, communication, sampling	Realistic expectations

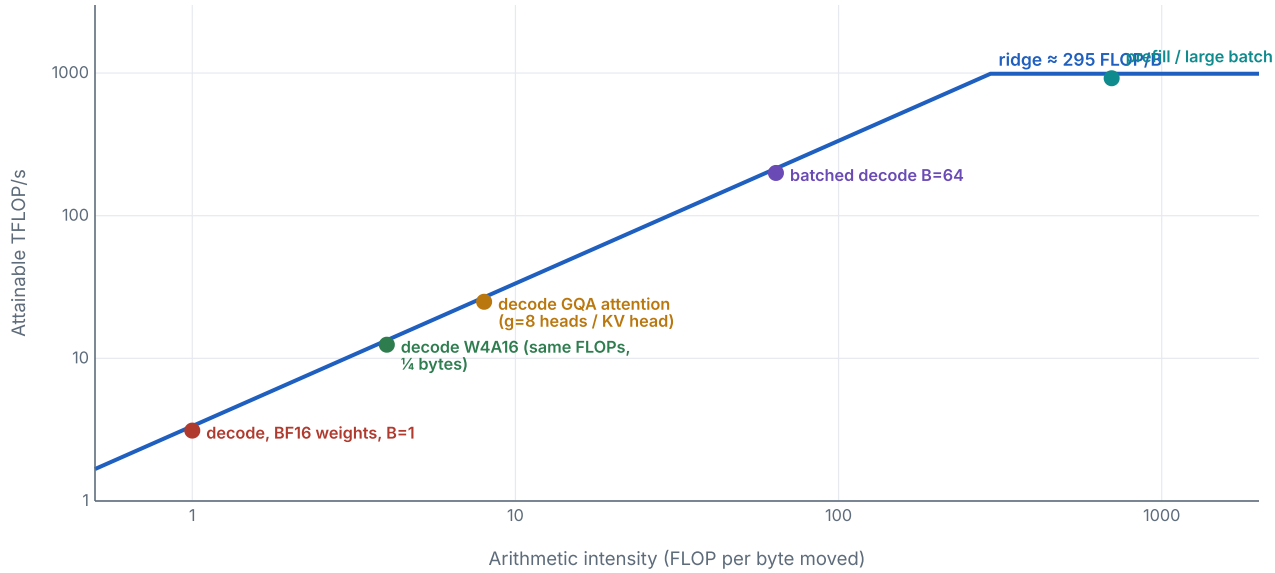


Figure 5.1 Roofline sketch with H100-class parameters used in P1 §14.1 (β ≈ 3.35 TB/s, peak ≈ 989 TFLOP/s ⇒ ridge ≈ 295 FLOP/B). *Illustrative placement*: points show where each regime lives, not measurements. Quantisation moves a decode point *right* (fewer bytes per FLOP) and up the memory-bound slope; batching and speculation do the same by reusing each weight byte more times.

Worked example A — which weights fit and how much KV is left DERIVED

70B dense model, Llama-3.1-70B-style KV (80 layers, 8 KV heads, d_h = 128): 2·80·8·128·2 B = 320 KiB per token at 16-bit (matches P1 §19.3). One 80 GB GPU, ignoring activations/fragmentation (several GB in practice):

Weights	Weight GB	Free GB	KV tokens @ 16-bit KV (320 KiB)	@ FP8 KV (160 KiB)	@ 4-bit KV (≈ 80–90 KiB)
BF16	140	—	does not fit one 80 GB device		
FP8 / INT8	70	≈ 10	≈ 30 k (< one 32 k sequence)	≈ 61 k	≈ 108–122 k
W4A16 (g128) ≈ 37 GB	37	≈ 43	≈ 131 k	≈ 262 k	≈ 470–525 k

Lesson (P1 §19.3): weight quantisation decides whether the model fits at all; KV precision then decides concurrency and context. Forge's capacity planner (SV-5) automates exactly this table and then subtracts measured engine overheads.

Worked example B — ideal decode ceilings on an H100-class part DERIVED

Ideal single-stream decode ≈ β / bytes-per-token with β = 3.35 TB/s (short context, KV negligible): 8B BF16 (16 GB) ≈ 4.8 ms → ≲ 210 tok/s; 8B W4A16 (≈ 4.5 GB) ≈ 1.3 ms → ≲ 740 tok/s; 70B W4A16 (≈ 37 GB) ≈ 11 ms → ≲ 90 tok/s. Real stacks land below these ceilings (attention/KV reads, dequantisation, launch overhead, sampling); the cost-model residual (§5.4) tells us by how much on each chip, and the KPI targets of §2.3 deliberately sit below the ceiling.

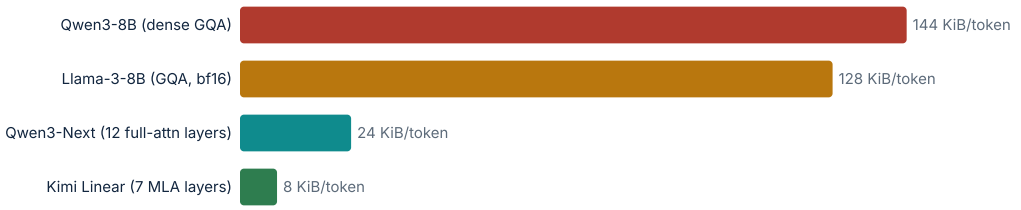


Figure 5.2 KV-cache bytes per token for four public configurations, as derived in P3 (Table 2.1 from public config files). Architecture, not post-hoc compression, accounts for most of the 5–18× spread — which is why the planner can recommend architecture changes when training budget exists.

5.2 Rule base: the papers' decision procedures as data

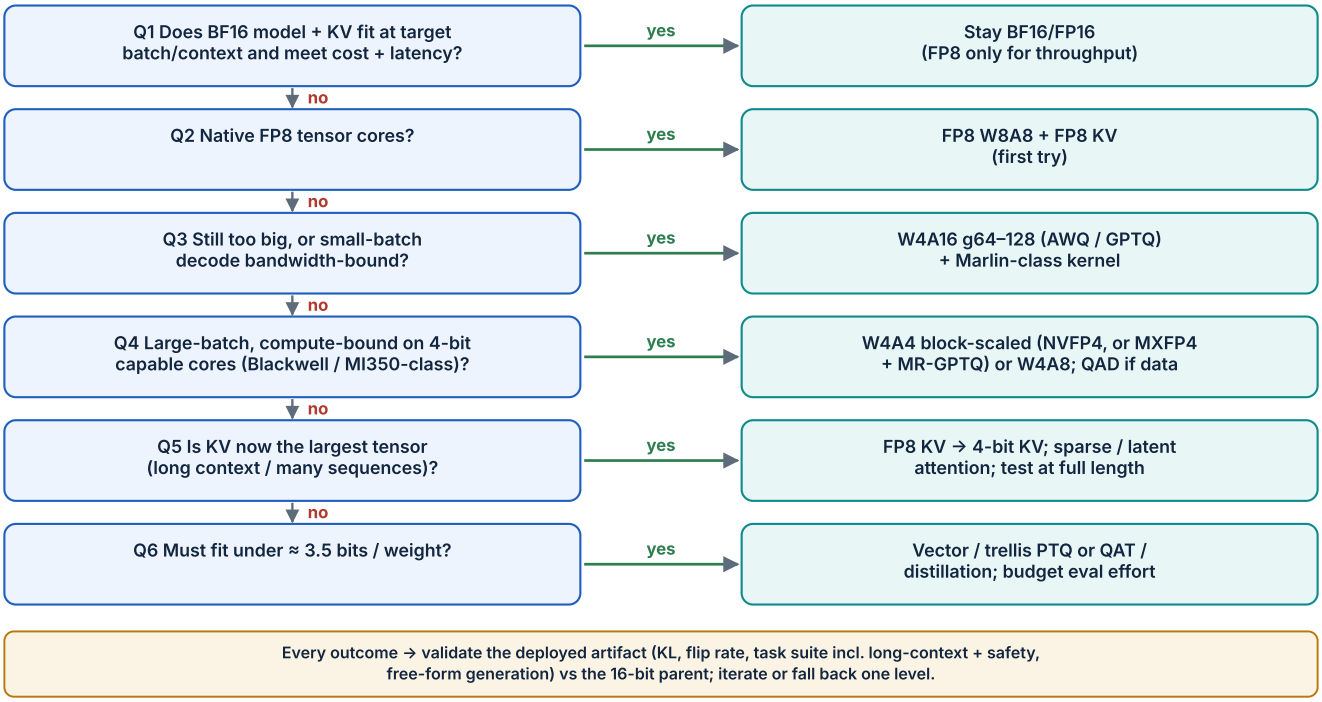


Figure 5.3 Quantisation decision flow, adapted from P1 Figure 19.1 (author's synthesis there; encoded here as the planner's first rule set). Questions are not mutually exclusive — a deployment may pass through several (e.g., FP8 weights/activations with a 4-bit KV cache).

Table 5.1 — Planner rules beyond quantisation (each rule links to its claims-ledger entries)

If ...	... the planner proposes (first-try order)	Source
Need a smaller dense model for any hardware; training budget available	Structured width pruning (respecting GQA groups) → logit/on-policy distillation from the unpruned teacher; ≥ several B, ideally tens of B tokens; compare with "distil into a fresh smaller student" arm	P2 §1.2, P4 §1.3
Serving a very large MoE, memory-bound	Router-aware expert pruning/merging calibrated on mixed domains incl. generative samples; then MoE-aware quantisation; expert-offload tiers	P2 F5
Decode-heavy dense inference, custom kernels allowed	Add training-free activation sparsity (TEAL-style 40–50%, 1.3–1.8× in the papers' kernels) on top of the quantised model	P2 F5
Many fine-tunes of one base	Store deltas (low-rank or 1-bit sign + scale); multi-tenant adapter serving	P4 §1.3

Ship same model in fewer bytes with *zero* behaviour change	Entropy-code weights (≈ 30% for BF16); decode in a fused kernel only if one exists for the chip	P4 F7
Long context, KV dominates, retraining unavailable	Paging + prefix caching + chunked prefill (lossless) → FP8 KV → 4-bit KV after full-length validation → conservative eviction only for single-query retrieval-style workloads	P3 F4–F5, P1 F8
Long context, training allowed	Evaluate MLA conversion, local/global patterns, hybrid linear layers, or trainable sparse attention with shared indexers — each against a same-token-budget dense baseline and multi-hop tests	P3 F2–F3, P6
Low concurrency, released drafter / MTP head available	Enable speculation, measure acceptance per position, tune γ; n-gram lookup first for context-copying workloads (code editing, structured agents)	P7 §19.3
High-concurrency throughput serving	Benchmark with speculation *off* first; adaptive γ including 0	P7 §5.8
Training a model from scratch / large continued training	FP8 default; NVFP4 only after the TR-2 diagnostics pass, with BF16 exemptions and a high-precision tail phase	P1 F7, P5
Low-rank on a frozen model	Whiten with calibration activations, allocate ranks heterogeneously, add short LoRA healing; measure wall-clock not parameter count	P4 §1.3

5.3 Search loop (pseudocode)

```
# plan(model, chip, workload, budget) -> ranked list of validated recipes
def plan(model, chip, workload, budget):
    desc   = hardware_descriptor(chip)              # measured, not datasheet (HW-2)
    binding = binding_resource(model, desc, workload) # capacity | bandwidth | compute | kv | accuracy
    cands  = rulebase.propose(model.form, binding, desc.formats, budget.allow_training)
    cands += baseline_arms(model, budget)            # incl. "smaller dense model of equal footprint" (P4 F8)
    cands  = [c for c in cands if fits(c, desc, workload)]   # byte accounting, ex. A
    scored = [(c, cost_model.predict(c, desc, workload)) for c in cands]
    scored = prune_dominated(scored)                 # drop recipes dominated on (bytes, latency, accuracy-
proxy)

    # successive halving: cheap proxies first, expensive evals only for survivors
    rung = scored
    for stage in [PROXY_KL, SUITE_SHORT, SUITE_LONGCTX_REASONING_SAFETY, END_TO_END_SERVING]:
        results = run_experiments(rung, stage, seeds=stage.seeds, stats=bootstrap_ci)
        rung    = keep_top(results, k=stage.keep, gate=budget.gates[stage], noise_aware=True)
        cost_model.calibrate(results)                # residuals update efficiency factors (5.4)
        claims.update(results)                       # ledger: reproduced / contradicted (5.5)
    return pareto_front(rung, axes=[bytes, p50_latency, throughput, quality_score, risk])
```

> **WHY SUCCESSIVE HALVING, AND WHY "NOISE-AWARE"**
>
> Full evaluation suites with long-context and long-generation tests are expensive (P1, P3). Cheap KL/flip-rate proxies eliminate most bad candidates; only survivors earn the costly stages. Because P5's audit showed most benchmark differences are within sampling noise, "keep_top" compares confidence intervals and refuses to rank statistically indistinguishable candidates — it keeps both and lets cost break the tie.

5.4 Calibrating the cost model

The physics are exact only for ideal kernels. The planner therefore carries **efficiency factors** η that are fitted from measurements and stored per (chip fingerprint, kernel class):

```
t_pred = max( bytes_mem / (eta_mem * beta_peak),          # bandwidth term
              flops     / (eta_mma * pi_peak[dtype]),     # tensor-core term
              dequant_ops / (eta_alu * alu_peak) )        # CUDA-core / vector ALU term (P1 eq. 14.1)
           + launch_overhead + selector_scan_bytes / (eta_mem * beta_peak)
residual   = t_measured / t_pred          # logged; eta fitted by robust regression per kernel class
alert      if residual outside [0.8, 1.25] for a shape bucket   # model or kernel is wrong: investigate
```

A residual alert is a *feature*, not a nuisance: it flags either a mis-modelled regime or an under-performing kernel. Both are valuable; the kernel lab (§9) shows residuals as a heat map over shape space.

5.5 The claims ledger

Each rule or default points at one or more `Claim` records, e.g. *"Marlin-class W4A16 keeps ≈ max 4× speed-up to batch 16–32" (P1 §14.2) — status: untested → reproduced on chip X at B ≤ 16 with 3.4× → contradicted on chip Y above B=8*. The ledger has three jobs: (1) it prioritises Phase-1 experiments by how many rules depend on a claim; (2) it lets the UI show provenance badges (UI-3); (3) it gives a defensible answer to "why did Forge choose this?". Claims from vendor material or single-source results (the papers' own tags: P5 [V]/[U], P6 "single-source", P7 [V]) start with a prior of *untrusted*.

6 Hardware Abstraction and Custom Kernel Layouts

The user requirement is "custom kernel layouts for optimal hardware interaction with any given chip, GPU or accelerator". The papers give us the physics (bytes are the currency; dequantisation must hide under HBM time; fused quantisers; selector scans matter) and several concrete designs (Marlin-style W4A16, block-scaled FP4 operand layouts, sparse-attention patterns). This chapter turns them into a **general method for deriving a kernel's memory layout and schedule from a chip descriptor**, plus a catalogue of the kernel families Forge needs.

6.1 Core idea: spec → layout → schedule

A fast kernel is the combination of three decisions that must be made *together* for a specific chip:

1. **Layout** — how bytes are arranged in memory so that each hardware thread issues full-width, aligned, conflict-free loads and receives exactly the operand fragments the matrix unit wants (no shuffles). Layouts are produced *offline* when the artifact is built, which is why they are part of the weight artifact (`layout_id`), not of the kernel.
2. **Schedule** — tile shapes, pipeline depth, work partitioning over compute units, split-K / split-KV policy, fusion boundaries.
3. **Numerics** — accumulation precision, where scales are applied, rounding mode, clamping.

Forge represents these in a **kernel spec** (Appendix C.2) and generates implementations from it. The spec is chip-independent; a **layout deriver** and the **autotuner** instantiate it for each chip descriptor. This is what "optimal for any given chip" means operationally: the derivation is automatic, the result is measured, and the outcome is cached by chip fingerprint.

6.2 What the chip descriptor drives

Descriptor field	Kernel decision it controls	Example consequence
Max global-load width (bytes/thread)	Packing granularity of reordered weights	128-bit ⇒ each thread's word holds 32 INT4 weights (W4 layout)
Matrix-unit native tile (M×N×K) and operand fragment layout	Fragment ordering inside the tile; minimum tile	Weights reordered so a thread's registers are filled by one load
Shared memory per compute unit; bank count/width	Pipeline stages × tile bytes ≤ capacity; XOR swizzle pattern	Stages = ⌊smem / (A+B+scales tile bytes)⌋, capped at 4–6
Register file per compute unit; max regs per thread	Accumulator tile size; dequant temporaries	Large accumulator tiles trade occupancy for reuse

Compute-unit count (SMs / CUs)	Persistent/striped partitioning for skinny decode shapes	Avoid wave-quantisation idle units at M=1–16
Supported dtypes in the matrix unit (FP16/BF16/FP8/FP6/FP4/INT8/INT4, block-scaled?)	Whether dequantise-to-FP16 or consume directly; scale tensor layout	Block-scaled FP4 consumed natively where supported; otherwise W4A16 path
Async copy / bulk-copy engines (e.g., tensor-memory-accelerator-class units)	Producer/consumer warp specialisation vs simple multi-stage ring	Hopper-class: bulk async + warpgroup MMA; older: cp.async rings
HBM bandwidth, L2 size, effective ridge (measured)	Split-K and activation-residency heuristics; roofline targets	Activation tile pinned in L2 across the weight sweep (Marlin)
Interconnect topology	MoE all-to-all and tensor-parallel overlap strategy	Quantised all-to-all only if link-bound
Cache line / transaction size	Alignment padding of scale tensors and KV pages	KV page size multiple of transaction size

6.3 Layout derivation algorithm (pseudocode)

```
def derive_layout(spec, chip):
    # 1. Operand fragment order: dictated by the matrix-unit instruction, not by taste
    mma   = chip.matrix_unit.best_for(spec.compute_dtype, spec.M_hint)      # e.g. 16x8x16 fp16, block-scaled
fp4 atom
    frag  = mma.operand_fragment_layout("B")                                 # which (n,k) elements each lane
owns

    # 2. Packing: make one vector load carry exactly what one lane needs for several MMAs
    vec_bytes   = chip.max_vector_load_bytes                                 # 16 on most GPUs
    elems_per_v = vec_bytes * 8 // spec.weight_bits                          # 32 for 4-bit
    perm        = interleave_for_fast_dequant(frag, spec.weight_bits, chip.alu_tricks)  # bit-interleave so
unpack = 1 mask+shift per 2 values

    # 3. Tile shape: maximise reuse under capacity limits
    for tile in candidate_tiles(mma, spec.tile_ranges):
        smem = stages * (bytes_A(tile) + bytes_B(tile) + bytes_scales(tile, spec.group))
        regs = accum_regs(tile) + dequant_temps(spec) + addressing
        if smem <= chip.smem_per_cu and regs <= chip.regs_per_thread_budget:
            keep(tile)

    # 4. Shared-memory swizzle: eliminate bank conflicts for the chosen access pattern
    swz = xor_swizzle(chip.smem_banks, chip.bank_width, access_pattern(frag, perm))

    # 5. Scale placement: contiguous along the axis consumed per MMA step; padded/swizzled if hardware demands
(e.g. 128x4 atoms)
    sc  = scale_layout(spec.scale_format, chip.scale_atom, spec.group)

    # 6. Work partition: stripe tiles over compute units so skinny-M decode keeps all units busy
    part = striped_partition(tiles=kept, units=chip.compute_units, split_k="auto")

    return Layout(id=hash(chip.family, mma, perm, swz, sc, tile), perm=perm, swizzle=swz, scales=sc,
partition=part)
```

The `layout_id` is written into the quantised artifact's manifest. A chip that cannot execute that layout efficiently triggers an *offline re-pack* (a cheap, bit-exact permutation) rather than a re-quantisation — which is why layouts are separable from the numerical content of the weights.

6.4 Kernel catalogue

K1 — Weight-only mixed-precision GEMM (W4A16 / W8A16, optionally W3/W6)

Regime: bandwidth-bound decode, batch ≲ 16–32 (P1 §14.2: Marlin remains near the ideal 4× to those batches, then decays gracefully to 64–128). **Why it exists:** it converts the 16→4-bit byte reduction into latency *only if* dequantisation is hidden under the HBM stream: with ≈ 3 ALU ops per weight, dequantisation time is a third of the HBM time on H100-class numbers, but poorly written kernels lose the gain (QServe reports 20–90% overhead for naïve INT4 dequantisation, P1 §14.1).

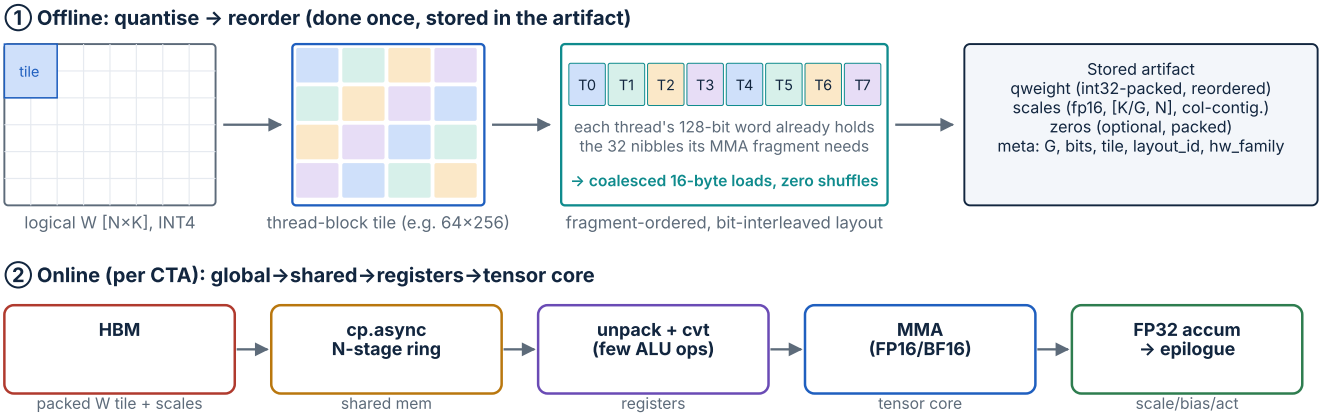


Figure 6.1 Weight-only INT4 GEMM: offline reordering makes each thread's 128-bit load carry its fragments in MMA order; online path hides dequantisation behind the async copy ring. Design elements follow the Marlin description in P1 §14.2.

Aspect	Specification
Layout	Weights: quantise → permute into fragment order → bit-interleave → pack into int32 lanes (16-byte vector units). Scales: fp16/bf16 per (group, output channel), stored column-contiguous so one transaction serves a warp. Optional zero-points packed with scales. Tile-aligned padding.
Schedule	Multi-stage async copy ring (2–6 stages by smem budget); activation tile resident in shared memory / L2 across the weight sweep; striped partition of tiles over compute units with deterministic reduction order for split-K; per-column scales applied once in the epilogue when mathematically equivalent (P1 §14.2).
Numerics	Dequantise via magic-number/bit-twiddle conversion to FP16/BF16 pairs; FP32 accumulation; documented tolerance vs the FP32 reference on random + captured activations.
Variants	FP8-weight-only on chips lacking FP8 matrix units; W4A8 (INT8 activations, QQQ/QServe-style progressive quantisation); 2:4-sparse + quantised combos; LUT kernels for non-uniform / vector-quantised weights (LUT-GEMM, FLUTE, T-MAC, AQLM/QuIP#/QTIP-specific codebook kernels, P1 §14.2); bit-plane layouts for 3/5/6-bit (FP6-LLM-style pre-packing, P1).
Tests / KPI	≥ 85% of measured HBM bandwidth at B ≤ 16; speed-up vs BF16 GEMM ≥ 3.2× at B=1–16; correctness within tolerance across K ∈ {not-multiple-of-group, odd}, M ∈ {1…256}.

K2 — Block-scaled FP4 GEMM (NVFP4 and MXFP4) with fused quantiser

Regime: compute-bound prefill/large batch and training GEMMs on chips with native 4-bit matrix units. **Why layout matters:** the tensor core expects one block scale per 16 (NVFP4) or 32 (MXFP4) elements along the contraction axis, delivered in a specific padded + swizzled arrangement; P5 §11.2 documents TN-layout-only

GEMM, row- and column-wise copies stored separately, padding of the scale tensor (first dim ×128, second ×4) and swizzling before the GEMM.

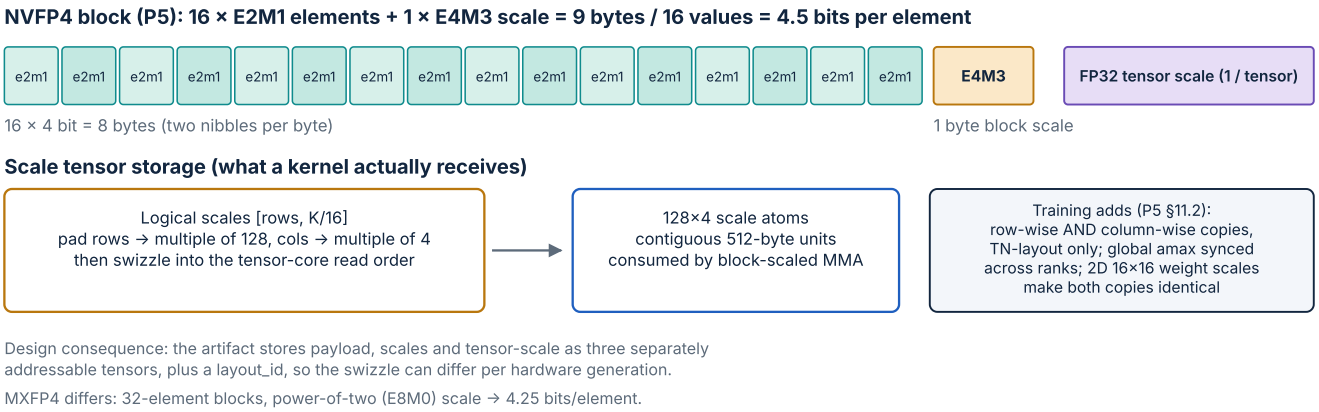


Figure 6.2 NVFP4 storage unit and scale-tensor handling (P5 §3.3, §11.2). The artifact stores payload, block scales and tensor scale as separate tensors so layouts can change by hardware generation without touching numerics.

Aspect	Specification
Fusion (mandatory)	Activation quantisation in the producing kernel's epilogue (e.g., RMSNorm/activation → absmax per 16 → E4M3 scale → E2M1 round → packed write) or in the GEMM prologue; unfused FP4 quantisation costs ≈ 59% of GEMM time at N=4096 and cuts a 3× peak ratio to ≈ 2.3× (P5 eq. 11.1, Table 11.2, derived).
Rounding	RTN for forward/inference; stochastic rounding (counter-based RNG, reproducible per tensor/step) for gradient operands with the scale headroom the unbiasedness proof needs (P5 §5.2: the 16/17 factor ⇒ ≈ 25% MSE cost on Gaussian data).
Scales	Per-tensor FP32 scale = amax/(448·6); per-block E4M3 scale = $(\text{amax}_{\text{block}}/6)/s_{\text{global}}$; global amax synchronised across ranks for gathered tensors; E4M3 range diagnostics (loses precision when a block max < 3.5×10−5 of the tensor's; flushes below 2.2×10−6, P5 key finding 4).
Training extras	Row-wise and column-wise operand copies (TN layout); optional 2D 16×16 weight scales so one quantisation serves both copies; Hadamard transform kernel (K8) on Wgrad inputs behind a flag, because P5 finds RHT does not reduce RTN error for NVFP4 but cuts SR-product variance 6.6–22× on spiky data.
Fallback	Chips without native 4-bit units dequantise FP4 payload to FP8/FP16 inside a K1-style kernel (weights only), keeping the storage win.
Tests / KPI	Bit-exact vs reference quantiser for RTN; statistical tests for SR (unbiased mean, expected variance); tile zero-gradient-fraction diagnostic (TR-2); ≥ 2.2× vs FP8 GEMM on compute-bound shapes where the hardware peak ratio is 2×.

K3 — FP8 GEMM (per-tensor, per-block) and INT8 W8A8

The production default where native FP8 exists (P1 decision flow Q2). Layout is conventional (row/column major with 128-byte alignment); the interesting parts are scaling granularity (per-tensor for weights, per-token or per-block 1×128 for activations in frontier practice), fused scale application, and FP8 KV write paths. INT8 W8A8 (SmoothQuant-style) remains the path for chips with INT8 but no FP8 units. **KPI:** ≥ 90% of vendor GEMM library throughput on large shapes; bitwise reproducibility across runs for inference.

K4 — Decode attention: paged, GQA-grouped, split-KV, quantised-KV, MLA

Regime: always bandwidth-bound in decode; arithmetic intensity is only about *g* FLOP/byte for *g* query heads per KV head (P3 F1), so the single most important layout rule is: **put all query heads of a GQA group in the same thread block so each KV byte read serves *g* heads**.

Aspect	Specification
KV page layout	Fixed-size pages (e.g., 16–64 tokens) with head-major layout inside the page so one head's keys are contiguous (coalesced) and value tiles match the second GEMM's fragment order; page size is a multiple of the transaction size; page table in device memory; radix-tree prefix sharing maps pages by block hash (P3 Ch. 9).
Quantised KV	Keys quantised per-channel and values per-token (KIVI-style, P1 F8) with a small full-precision residual window of recent tokens; scales stored in side tensors co-located per page; dequantisation fused into the QKT and PV stages. Default FP8; 4-bit gated by long-context evaluation.
Parallelism	Split-KV (flash-decoding-style) across compute units when batch × heads is smaller than the unit count; two-pass reduction with a numerically stable log-sum-exp merge.
MLA variant	Cache the compressed latent (e.g., 576 values/token/layer in DeepSeek-V3, P3 F2); absorb up-projections into Q and output projections so decode attends in latent space; layout stores latent and RoPE parts separately.
Hybrid models	Only full-attention layers own paged KV; linear/SSM layers own fixed-size state snapshots placed deliberately for prefix reuse (P6 §12.7 item 5).
KPI	≥ 85% of bandwidth for read-dominated shapes at 32K–128K context; exact (tolerance-bound) agreement with reference softmax attention.

K5 — Sparse-attention kernels: indexer, top-k, gather-attention

P6's byte accounting shows that in a V3.2-style layer at 128K context the **index scan consumes 88% of bytes read**, and that sharing the scan across 4 layers lifts the byte reduction from 7.7× to 22.6× (57× if one indexer served all 61 layers, derived there). Hence the kernel design priorities are:

- **Cheap selector bytes**: index keys in FP8/FP4, few indexer heads, hierarchical or shared indexers (P6 §12.7 items 1–2). The indexer kernel is a bandwidth-bound scan; use the K4 page layout with a separate, narrower *index-key page*.
- **Contiguous reads**: select at block or page granularity and share the selection across the heads of a GQA group to preserve arithmetic intensity (P6 §12.7 item 1). P6 also proves block selection cannot capture more attention mass than token selection at equal budget — so block size is a bandwidth-vs-recall trade-off the autotuner must expose.
- **Prefill loop order**: KV-outer loops or gathered queries to fill tensor-core tiles (item 3). Overlap selection with data movement of the previous layer's compute (item 4).
- **Top-k selection**: radix/bitonic partial sort in shared memory over the indexer scores, producing a compact index list consumed by the gather-attention kernel.

K6 — Speculative-decoding kernels

From P7 §6 and §12.1: (i) **tree attention** with custom mask and per-node position ids (kernels with non-coalesced access patterns can dominate the critical path); (ii) **fused verification** — compute target and draft probabilities, run the recursive rejection-sampling rule, and emit accepted tokens plus the bonus token without CPU round-trips (sampling time is < 1.7% in one measured engine, but CPU launch overhead is 3–12%, so the

draft loop must be CUDA-graph-capturable); (iii) **KV accept/rollback compaction** over ragged per-sequence accept lengths, preferably via the paged layout instead of copying; (iv) a drafter forward that consumes the target's hidden states (EAGLE-style), so the verification pass must expose them.

K7 — MoE kernels

Piece	Specification
Routing + dispatch	Fused softmax/top-k + token permutation producing contiguous per-expert token slabs; stable, deterministic ordering for reproducibility.
Grouped GEMM	One launch over all experts with a tile scheduler handling highly variable per-expert M (from 0 to hundreds of tokens); expert weights in K1/K2/K3 layouts; small-M experts use the K1 skinny path.
Expert-parallel comms	All-to-all with optional FP8/FP4 payload quantisation; overlap dispatch/combine with compute of other micro-batches; hierarchical (intra-node then inter-node) when topology says so.
Capacity features	Expert offload tiers (HBM → host → NVMe) with prefetch from router lookahead; pruned/merged experts handled by an indirection table so the same kernels run unchanged.
Caveat	P7 notes MoE can invert speculation's batch dependence; P1 rates MoE quantisation "young" — both flagged in the claims ledger as high-priority to measure.

K8 — Utility kernels

- **Fast Walsh–Hadamard transform** (block 16/32/128/head-dim): butterfly in registers/shared memory, fused with quantise-on-write; needed by rotation-based PTQ (QuaRot/SpinQuant-style) and the NVFP4 Wgrad recipe (P5 §6.6 gives costs: it adds a 16×16 matrix product per operand and is itself memory-bound).
- **Quantise / dequantise kernels** with a counter-based RNG for stochastic rounding.
- **Activation-sparsity gather-GEMV** (TEAL-style): per-token magnitude threshold ⇒ skip weight columns ⇒ gather only the surviving columns; reported 1.3–1.8× decode speed-ups at 40–50% sparsity (P2 F5) and composes with K1.
- **Fused entropy-decode + GEMM prologue**: decode exponent-coded BF16/FP8 weight blocks (Huffman/ANS-style) into shared memory then feed the MMA; DFloat11-style results are ≈ 70% size with bit-identical outputs (P4 F7). Worth it only where bandwidth-bound; otherwise decode once at load.

K9 — Low-rank and multi-adapter kernels

Low-rank layers are two thin GEMMs (xA then $\cdot B$) that save memory traffic only if rank $r < mn/(m+n)$ (P4 F5); fuse them to avoid materialising the rank-r intermediate in HBM. For fleets: a **segmented-batched LoRA/delta kernel** (one launch applies per-request adapters to a mixed batch) is required by the multi-tenant requirement SV-7; 1-bit-sign deltas (BitDelta-style) use a K1-like bit-packed layout with per-tensor scale.

6.5 Chip-family mapping

This table says what to *try* per family. Every "native" entry is a hypothesis to be confirmed by the capability probe (HW-2); nothing is enabled on the strength of this table alone.

Family	Typically available matrix-unit formats	Layout / schedule emphasis	Notes
NVIDIA Ampere-class	FP16/BF16, INT8, 2:4 sparse	cp.async multi-stage rings; fragment-ordered INT4 (original Marlin target); no FP8 units ⇒ FP8 is weight-only via dequant	Baseline for K1; widely deployed

NVIDIA Hopper-class	+ FP8	Bulk async copy + warpgroup MMA, warp specialisation; mixed-input GEMM; FP8 KV default	P1 cites vLLM mixed-input work for Hopper
NVIDIA Blackwell-class	+ block-scaled FP4 / FP6 / FP8 (NVFP4, MXFP4)	128×4 scale atoms, TN-only FP4 GEMM, quantiser fusion; separate row/col operand copies for training	P5: SM 10.0+/10.3; peak FP4:FP8 = 2× (B200) or 3× (B300) vendor-reported
AMD Instinct CDNA3-class	FP16/BF16, FP8, INT8	Wavefront-64, LDS tiling, MFMA fragment layouts; HIP/CK-class kernels; different swizzle and bank rules	Same specs, different deriver tables
AMD Instinct newer (FP4/FP6-capable)	+ MX formats (MXFP4/6)	MXFP4 (32-element, power-of-two scale) path; AMD training results reported in P5	P1 groups with “MI350-class”
CPU / Apple silicon / edge	Vector ISA (AVX/NEON), matrix extensions where present	LUT-based low-bit kernels (T-MAC-style), GGUF-class block formats, cache-blocked GEMV	Deployment target for small/quantised models
Systolic-array accelerators (TPU-class and similar)	BF16, INT8, FP8 on newer	Compiler-expressed kernels (XLA/Pallas-class); tiles matched to array shape (e.g., 128-multiples); layouts expressed as padding/transposes rather than thread-level fragments	Tier B/C until a descriptor and probe exist
Unknown / new accelerator	—	Run probe + micro-benchmarks; fall back to portable kernels (Tier C); autotune on the generic spec; file a descriptor	NFR-V1: degrade, never fail

6.6 Autotuning and measurement protocol

1. **Search space**: tile (M,N,K) multiples of the MMA atom; stages 2–6; split-K ∈ {1,2,4,8,auto}; page size; group size (if the spec allows); vector width. Infeasible points are discarded analytically (smem/register equations of §6.3).
2. **Model-guided pruning**: rank the remaining candidates with the cost model (§5.4) and measure only the top N plus a random sample (to catch model errors).
3. **Measurement hygiene**: lock clocks where possible; warm up; flush L2 between iterations for bandwidth-bound kernels; report median and inter-quartile range over ≥ 30 runs; record temperature/power state; detect throttling.
4. **Correctness gate**: each candidate is compared with the reference oracle on random and *captured real* activations; integer paths must be bit-exact, floating paths must meet per-dtype tolerances; failures are never cached.
5. **Cache key**: (chip fingerprint, driver/compiler version, op, dtype config, shape bucket, layout_id). Shape buckets follow powers of two plus “model-native” shapes.
6. **Regression CI**: stored baselines per kernel × chip; alert if the median drops > 5% or the residual (§5.4) leaves its band.

6.7 Fusion map and numerical edge cases

Fusion boundary	Rationale
RMSNorm + activation quantise (FP8/FP4 absmax)	Avoids an HBM round trip of the BF16 activation (P5 §11.3).
GEMM epilogue: scale + bias + activation + residual + quantise-for-next-GEMM	Output of one GEMM is the quantised input of the next; fusion makes the quantiser “free” (the upper-bound case in P5 Table 11.2).
RoPE + KV quantise + KV page write	One pass over K/V before they are cached.
Attention output projection ± adapter delta	Keeps fleet adapters from adding a separate kernel launch per layer.
Logits → sampling → rejection verification	Removes CPU round-trips in speculation; sampling cost is small but launch gaps are not (P7 §12.1).

EDGE-CASE TEST LIST (MUST PASS BEFORE A KERNEL IS “DONE”)

Non-multiple-of-group K; M=0 experts; ragged batches; GQA group sizes 1, 2, 4, 8, 16; sequence length not multiple of page size; E4M3 scale underflow/flush blocks and blocks with all-zero values; channels 10–30× smaller than neighbours inside 16×16 weight tiles (zero-gradient failure, P5); NaN/Inf propagation conventions of each format (E4M3 “FN” convention); counter-based RNG reproducibility across launches; deterministic reduction order for split-K when bit-reproducibility is requested.

7 Model Forms, Training and Deployment Plans

This chapter specifies how Forge treats each of the three forms, how training and distillation work under the precision ladder, and how artifacts are served and rolled out. Every "reference recipe" below is a *starting hypothesis* from the papers' decision guides; the planner (§5) and Phase-1/2 experiments decide what actually ships.

7.1 Dense models

Reference recipes

ID	Situation	Recipe (stage order)	Why this order
D1	Native FP8 hardware; throughput focus; no training budget	FP8 W8A8 → FP8 KV → paged KV + prefix cache + chunked prefill → (low batch only) speculation	FP8 W8A8 was effectively lossless in the Llama-3.1 study cited by P1; lossless serving features add on top at zero accuracy risk.
D2	Large model must fit one GPU; small-batch decode	Calibrate → AWQ/GPTQ W4A16 (g64–128) → Marlin-class kernel → FP8 KV → validate at target context	Bandwidth-bound: bytes are the lever (P1 F1–F2). Weight compression decides fit; KV precision decides concurrency (Example A).
D3	Need a smaller model; training budget exists	Importance estimate → structured width prune (GQA-aware) → logit / on-policy distil from parent (tens of B tokens) → W4A16/FP8 → validate; run "fresh student" baseline arm in parallel	Most reliable route to a smaller dense model (P2 §1.2); distillation must beat a train-from-scratch small model (P4 F8).
D4	Long-context serving; training allowed	Candidate set: GQA→MLA conversion; local/global layers; hybrid linear layers; trainable sparse attention with shared indexer. Each trained against a same-budget dense baseline, evaluated on multi-hop + RULER/NoLiMa at full length; then FP8→4-bit KV	Architecture does most KV work (P3 F2) but hybrids showed multi-hop deficits (P3 F7); sharing the indexer is reported nearly free in three papers but not independently replicated (P6 §13.6).
D5	Blackwell-class throughput; reasoning model	BF16 parent → NVFP4 PTQ (MR-GPTQ-style) → if reasoning loss: QAD from BF16 teacher → validate long generation	NVFP4 PTQ recovers 97–99% for ≥ 30B but RL-trained reasoning models lose most under task-loss QAT; distillation-based QAT recovers them (P5 §19.1).
D6	Cheap extra decode speed on a quantised dense model	Add TEAL-style activation sparsity 40–50% via gather-GEMV; re-validate	Training-free, composes with quantisation (P2 F5); validate reasoning since depth-/activation-style pruning hurts generation more than MCQ (P2 F4).

THINGS THE DENSE TRACK DELIBERATELY DOES *NOT* PRIORITISE

Unstructured pruning for GPU speed (rarely yields wall-clock gains, P2 F2); 2:4 sparsity unless sparse fine-tuning and a cuSPARSELt-class stack are available and generative tasks are validated (P2 §1.2); post-hoc low-rank as a standalone shrinker (niche, P4); training-free KV eviction as the primary long-context answer (fails multi-hop and long CoT, P3 F4); ternary/binary native models (lowest ranked in P1's priority table).

7.2 Mixture-of-Experts models

What changes relative to dense

- **Binding resource:** total parameters (every expert must be resident or offloadable) and all-to-all communication, not per-token FLOPs. Active parameters per token are small, so single-stream decode touches fewer bytes, but at realistic batch sizes most experts are touched each step.
- **Compression targets:** expert count (pruning/merging), expert weights (quantisation), and routing-aware calibration. REAP is reported near-lossless for code at 50% expert pruning on Qwen3-Coder-480B and Kimi-K2 (P2 F5, checked against abstracts there); MoE quantisation is "young" and rated Tier 3 by P1.
- **Speculation:** can invert its dependence on batch size (P7 §5.8) — the simulator must model expert-touch counts per verification step.

Expert-pruning/merging pipeline (pseudocode)

```
stats  = collect_router_stats(model, calib_mix)         # per-expert: usage freq, gate weight mass, output
norm; incl. GENERATIVE samples (P2 F5)
score  = expert_saliency(stats)                           # router-weighted activation contribution (REAP-
style)
plan   = choose_experts(score, keep_ratio=r, per_layer_floor=min_k, domain_floors=domain_stats)
model2 = prune_or_merge(model, plan)                      # merge = weight-average clustered experts, update
router logits
model2 = brief_router_finetune(model2, calib_mix)         # optional, small token budget
report = eval(model2, suites=[code, chat, long_ctx, safety, tail_domains])   # tails matter: rare-expert
knowledge
# gate: no domain falls below parent - tolerance; otherwise raise keep_ratio for that layer/domain
```

Parallelism planner

Given chip topology and the form, the planner enumerates (TP, EP, DP, PP) factorisations, estimates per-step communication from byte counts (all-to-all volume = tokens × hidden × bytes × 2 per MoE layer, halved if the payload is FP8), and selects by modelled step time subject to memory fit. Quantised all-to-all is enabled only when the interconnect, not compute, is the bottleneck.

MoE training notes

FP8 mixed precision is production-proven; FP4 for MoE is flagged open by both P1 and P5 (an independent 1T-token comparison reports gaps below 1.2% dense and 0.81% MoE with a refined recipe, but all-layer NVFP4 diverged in at least one setting, P5 §19.2). Forge therefore treats NVFP4 MoE training as a *gated experiment* (G4), never a default.

7.3 Scaled multi-agent fleets

> **STATUS OF THE EVIDENCE**
>
> The seven papers contribute the *building blocks* of an efficient fleet — prefix caching (top-tier in P3's priority list), delta/adapter serving (P4: BitDelta reports > 10× per fine-tune), context folding and typed eviction for agents (P3 F6), n-gram speculation for context-copying agents (P7 §19.3), disaggregated prefill/decode (P3 F5). They do **not** evaluate end-to-end agent fleets. This section is therefore a design built from those pieces, with explicit hypotheses and a gated evaluation track (Phase P10).

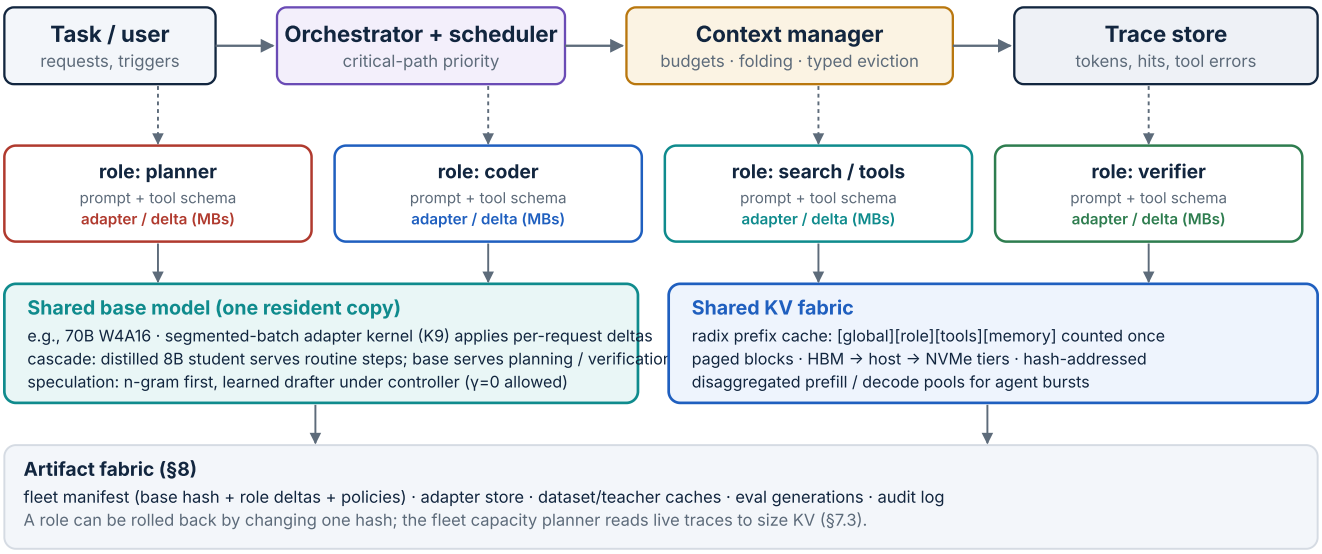


Figure 7.1 Fleet architecture. Agents are *roles* (system prompt + tool schema + adapter/delta) over a shared base model. Everything that can be shared (weights, prefix KV, draft models) is shared; everything private (task context) is budgeted and foldable.

Design elements and reasoning

Element	Design	Reasoning
Fleet manifest	Base artifact hash + per-role adapter/delta hashes + tool schemas + context policy + budget + routing rules; content-addressed like any artifact.	Reproducible fleets; roll back a role without touching others.
Prefix-friendly prompts	Canonical ordering: [global system] [role prompt] [tool schemas] [shared memory snapshot] [private task]. A linter flags prompts that put volatile text before stable text.	Radix prefix caching only helps if stable text comes first; cache hits cost zero prefill.
Adapters and deltas	Low-rank adapters or 1-bit-sign deltas per role; segmented-batched kernel (K9) applies them in mixed batches.	One resident base instead of N copies: memory scales with Σ deltas (MBs) not Σ models (GBs).
Model cascade	Small distilled student for routine steps (formatting, tool-call parsing, retrieval summarisation), large model for planning/verification; routing by learned or rule-based policy with a quality gate.	Compresses the average cost per step; student comes from recipe D3.
Context manager	Per-agent token budgets; triggers for folding/summarising old turns; typed, deterministic eviction (e.g., never evict tool results referenced by open plan items).	P3 F6: moving from text to agent *policy*; unbounded context is the main fleet memory risk.
Speculation	N-gram/prompt-lookup drafting first for agents that quote tool output; learned drafter for free-form text; controller disables under load.	P7 §19.3: n-gram lookup first for context-copying workloads.
Scheduler	Prefill/decode disaggregation to keep long agent prefill bursts from stalling decode; priority by critical-path (an agent blocking others goes first); preemption with KV swap to host tier.	Agent graphs have dependency structure that FIFO ignores.
Observability	Trace per task: agent graph, tokens per step, cache hit rate, fold events, tool errors, end-to-end success.	Needed to attribute a task failure to compression vs orchestration.

Fleet capacity formula and example DERIVED

KV for a fleet = $k \cdot (T_{\text{shared}} + \Sigma_{\text{agents}}\ T_{\text{private,i}})$, where shared prefixes are counted once. Example: Llama-3-8B-class KV (128 KiB/token, P3), 200 concurrent agents, each with a 24,576-token shared prefix and 8,192 private tokens. Naïve: 200 × 32,768 × 128 KiB = **800 GiB**. With prefix sharing: 3 GiB + 200 GiB = **203 GiB**. With FP8 KV: **≈ 101 GiB**. The fleet capacity planner computes this from live traces instead of assumptions.

Hypotheses tested in Phase P10

ID	Hypothesis	Pass criterion (initial)
H1	Prefix caching + canonical prompts cut fleet prefill compute by ≥ 60% on a representative agent workload.	Measured hit-rate ≥ 70% and TTFT improvement ≥ 2×.
H2	Per-role low-rank/1-bit deltas retain ≥ 98% of full-fine-tune task success.	Task-success CI overlaps parent; memory per role < 1% of base.
H3	FP8 KV and 4-bit weights do not reduce end-to-end task success beyond noise on multi-step tool-use tasks.	Success-rate difference CI includes 0; failure taxonomy shows no new error class.
H4	A cascade (student for routine steps) lowers cost ≥ 40% at ≤ 1-point success loss.	Pareto point on cost vs success.
H5	Context folding at a threshold keeps success within noise while bounding per-agent KV.	Max KV per agent ≤ budget; success within CI.

7.4 Training and distillation system

Precision ladder

Rung	Config	Entry gate	Diagnostics (all rungs log)
R0 BF16	Reference run, short horizon, fixed seeds	—	loss, grad norm, update/weight ratio, activation outlier stats
R1 FP8	FP8 mixed precision for GEMMs; BF16 for norms, embeddings, selected layers	Short-horizon loss within CI of R0	amax history, saturation fraction, loss gap vs R0
R2 NVFP4 (gated)	Block-scaled FP4 GEMMs; BF16/FP8 exempt layers; SR on gradients; RHT on Wgrad optional; 2D weight scaling optional; late BF16/FP8 “healing” phase	R1 stable; TR-2 diagnostics green on a pilot; hardware with native FP4	zero-gradient tile fraction, E4M3 scale occupancy, SR bias test, relative loss gap (published: ≈ 1–2% vs FP8/BF16, partially closed by a late high-precision phase, P5)

Workflow table

Workflow	Data needed	Token / sample budgets reported in the papers	Key controls
Prune → distil	Pretraining-like mix; teacher logits (cached top-k or live)	Tens to a few hundred B tokens (P2 F3); Nemotron Elastic: 9B and 6B nested models from a 12B parent with 110B tokens (P2, checked against abstract)	Importance estimator; GQA group integrity; width vs depth ratio; fresh-student baseline arm
On-policy distil	Prompts; live teacher scoring of student samples	Example: ≈ 70% AIME’24 from 60% SFT start in ≈ 150 steps; Qwen3 on-policy stage 1.8k vs 17.9k GPU-hours for RL (P4 F1, public write-ups)	Teacher/student style compatibility (P4 F2); capacity gate: small students can fail to learn from very strong reasoners

Reasoning-trace SFT	Teacher traces (licensed)	≈ 800k samples for R1-distilled family; ≈ 1k curated traces induced long-reasoning style in s1/LIMO (P4 F2)	Licence gate (TR-8); diversity monitor vs model collapse
QAT / QAD	Calibration/training mix; BF16 teacher	Method-dependent; QAD preferred for RL-trained reasoning models (P5)	Fake-quant exactness vs deployed kernel (bit-match test)
Sparse-attention adaptation	Long-context mix at target length	Hundreds of B tokens for DSA/SALA-style (P6 §14.2); none for selector-only reorganisation (HISA/MISA)	Same-budget dense baseline; indexer KL objective; multi-hop evals
Draft-model training	Target hidden states + token data	Recipe described in P7 Ch. 8 (no budget figure relied on here; measure in Phase P5)	Acceptance-per-position; training-time-test setup

DISTILLATION IS A LEGAL AND SAFETY SURFACE (P4 F3)

Terms-of-service violations, extraction campaigns, hidden-signal trait transfer and model collapse from recursive synthetic data are first-order risks. Forge's policy gate (TR-8) blocks jobs whose teacher licence forbids it; records teacher identity/hash in lineage; mixes non-synthetic data and tracks diversity metrics (n-gram entropy, self-BLEU, distinct-n) across generations of students; and runs paired safety evaluations of student vs teacher.

Training infrastructure requirements

- Parallelism: FSDP/ZeRO-class sharding, tensor/pipeline/expert parallel as the planner requires; deterministic data-loader order from manifests; global-amax synchronisation for FP4/FP8 (P5 §11.2).
- Checkpointing: asynchronous, sharded, chunk-deduplicated and delta-chained (§8.3); resume must reproduce data order and RNG state; keep a "pre-anneal" checkpoint and the best-validation checkpoint (retention policy §8.7).
- Failure handling: automatic restart from last good checkpoint; NaN/Inf and loss-spike detectors; FP4-specific alarms (TR-2).
- Cost and energy accounting per run and per token (HW-6, NFR-O1).

7.5 Inference and deployment

Serving topology options

Topology	When	Notes / source
Single device	Model + KV fit after quantisation; latency-sensitive	Removes inter-GPU communication from the critical path (P1 §2.1: 70B BF16 → ≈ 37 GB moves a two-GPU replica to one).
Tensor-parallel node	Larger models; high concurrency	Quantised collectives where link-bound.
Disaggregated prefill/decode	Mixed long-prefill and latency-sensitive decode (agents, RAG)	Lossless w.r.t. outputs; adds KV transfer cost (P3 F5).
Expert-parallel cluster	Large MoE	Needs all-to-all tuning (K7).
Workstation / edge	Local use; small or heavily compressed models	GGUF-class export; LUT kernels (K-table 6.5).

Speculation controller (pseudocode)

```
def choose_gamma(req, engine_state, models):
    a  = acceptance_estimator.per_position(req.workload_class)       # EMA of accepted/proposed at each draft
depth
    c  = draft_cost_ratio(models)                                    # drafter time / target step time
    B  = engine_state.running_batch
    best, best_gain = 0, 1.0                                         # gamma = 0 (no speculation) is always
allowed
    for g in range(1, MAX_GAMMA + 1):
        E_tok  = expected_tokens(a, g)                               # P7 closed form (geometric/position-
dependent)
        t_step = verify_time(B * (g + 1), engine_state.kv_bytes) + g * c * draft_unit_time(B)
        gain   = E_tok / (t_step / base_step_time(B))                # modelled speed-up vs autoregressive at
this batch
        if gain > best_gain * 1.03: best, best_gain = g, gain         # 3% hysteresis to avoid flapping
    return best                                                      # exactness guaranteed by rejection
sampling (P7 §3 and App. A.1)
```

KV block manager (pseudocode)

```
class KVBlockManager:                        # paged KV with prefix sharing, tiers and rollback
    alloc(seq, n_tokens)  -> block_ids       # free list per tier; evict by policy if HBM full
    match_prefix(tokens)  -> (block_ids, n_hit)   # radix tree keyed by rolling block hash (P3 Ch.9)
    fork(seq)             -> seq'            # copy-on-write for beam / speculation trees
    commit(seq, accepted) -> None            # speculation: keep accepted path, free rejected positions (P7
12.1)
    demote(block)         -> tier+1          # HBM -> host DRAM -> NVMe -> remote; async, prefetch on predicted
reuse
    promote(block)        -> tier-1
    stats()               -> {hit_rate, bytes_by_tier, fragmentation, rollback_bytes}
```

Release process

1. **Gate**: artifact passes evaluation gates (§10.1) and capacity plan (SV-5).
2. **Shadow**: replicate live traffic to the candidate, compare outputs (KL, flip rate on production prompts) without serving them.
3. **Canary**: 1–5% of traffic with automatic rollback on SLO breach or quality-metric regression.
4. **Ramp**: stepwise to 100%; keep the previous artifact warm for one release cycle.
5. **Monitor**: continuous sampled evaluation on live traffic (quality drift), cache hit rate, speculation acceptance, tail latency, cost per million tokens.

8 Data, Weights and Log Storage: Process Plans

A research system that trains, quantises, prunes, distils and evaluates dozens of variants will, left alone, drown in near-duplicate files: ten checkpoints per run × hundreds of runs × several precisions × teacher outputs × generations. This chapter specifies how every kind of data is stored and accessed **compactly, reproducibly and fast**. The unifying idea is simple: **store each distinct piece of information once, address it by its content, and describe everything else as a recipe over those pieces**.

8.1 Principles

1. **Content addressing.** Every object (chunk, tensor file, manifest) is named by its BLAKE3 hash. Identical bytes — across runs, checkpoints, variants, or users — are stored once.
2. **Derived-not-duplicated.** A quantised, pruned or re-packed model is a *derived artifact*: stored as bytes plus a recipe hash that can regenerate it. Cold-tier derived artifacts may be evicted and regenerated on demand, if the recipe and inputs are pinned (determinism requirement NFR-R1).
3. **Lossless where free, lossy only by recipe.** Storage-level compression (dedup, delta chains, entropy coding) must be bit-exact. Lossy steps (quantisation, pruning) are modelling decisions recorded as recipes, never silent storage tricks.
4. **Tiering.** Hot / warm / cold tiers are caches over one address space (Figure 8.1); promotion and demotion never rename objects.
5. **Small, mutable-looking metadata over large immutable data.** Manifests and the lineage DB change; blobs never do.
6. **Everything needed to recompute a metric is retained** (e.g., all generations), so later analyses do not need GPUs.

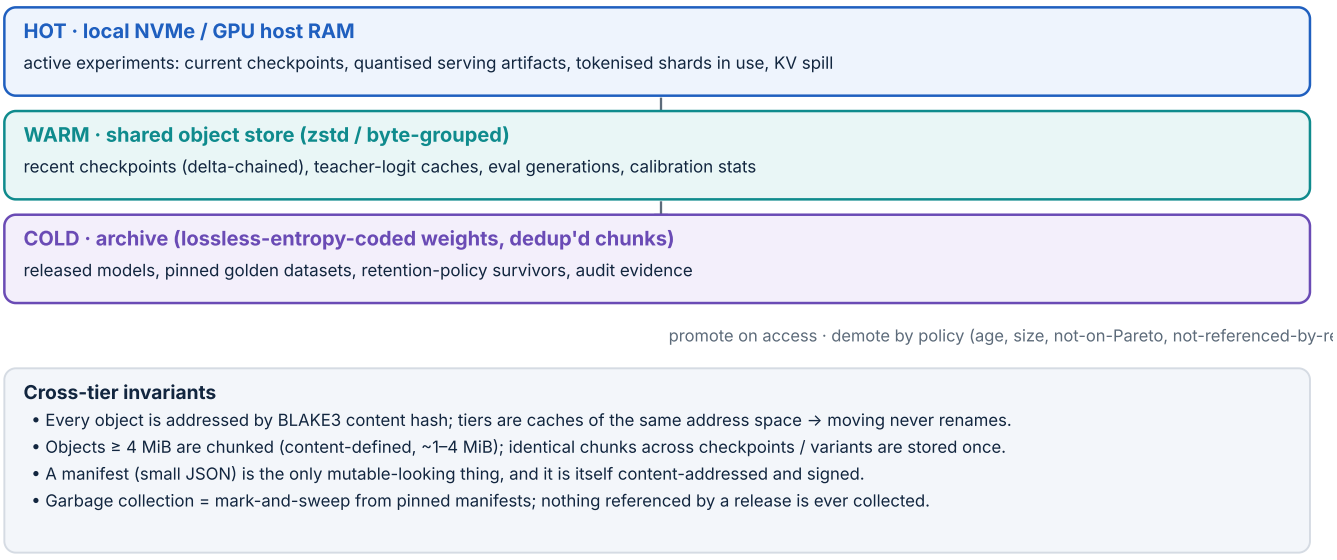


Figure 8.1 Storage tiers over one content-addressed address space.

8.2 Model weights: master + derived variants + deltas

Layout of a model artifact

Part	Content	Why
Tensor files	Safetensors-style header + aligned, memory-mappable tensor payloads, split into ≈ 1–4 GiB shards; each tensor independently addressable	Zero-copy loading; partial loading by tensor; dedup granularity matches tensors.
Quantisation sidecars	Scales, zero-points, block scales, tensor scales, codebooks, permutations — as *separate tensors* next to their payloads	Layout changes (re-pack) or scale refinement touch only sidecars, so unchanged payload chunks dedup (NVFP4 example, Fig. 6.2).
Manifest	Architecture config; tensor inventory (name, dtype, shape, hash, layout_id); recipe hash; parent hash; calibration-set hash; licence; hardware families validated on	The artifact is fully described without opening a blob.
Eval attachments	Hashes of EvalReports that gate this artifact	A weight file can never be "released" without its evidence.

Size budget for a 70B-class project DERIVED

Artifact	Bits / weight	≈ Size (70B params)	Role
BF16 master	16	140 GB	Source of truth; cold-tier entropy-coded to ≈ 98 GB (≈ 70%, bit-exact; DFloat11-class, P4)
FP8	8 (+ scales)	≈ 70 GB	Serving on FP8 hardware; exponent coding saves a further 10–15% (ECF8, P4)
W4A16 g128	≈ 4.1	≈ 36–37 GB	Single-GPU serving (P1 §2.1)
NVFP4	4.5	≈ 39 GB	Blackwell-class serving; MXFP4 would be 4.25 ⇒ ≈ 37 GB
Per-role delta (low-rank or 1-bit sign)	≈ < 1 (BitDelta reports > 10× vs a full fine-tune)	MBs–low GBs	Fleet roles / multi-tenant (P4)

Four variants of one model therefore cost ≈ 140 + 70 + 37 + 39 = 286 GB uncompressed, versus ≈ 98 (entropy-coded master) + regenerated-on-demand variants in the cold tier. The hot tier keeps only the variants a live deployment uses.

Lossless coding of weights: where and how

- **Why it works:** BF16 exponents carry far fewer than 8 bits of information (≈ 2.6 bits in DFloat11's analysis, P4), so a byte-grouped, exponent-focused entropy coder removes ≈ 30–33% (ZipNN ≈ 33%, DFloat11 ≈ 70% of original size). FP8 exponent concentration (ECF8) yields 9.8–14.8% on LLMs. The gain is bounded — it is a **storage and distribution** optimisation first.
- **Storage path:** chunks are encoded by tensor dtype (exponent stream and sign-mantissa stream separated, then entropy-coded; zstd fallback for other dtypes). Decoding is streaming and parallel on load.
- **Serving path:** decode once at load. Decode-inside-GEMM (fused kernel K8) is an *option* only where a bandwidth-bound workload and an existing kernel justify it (ZipServ-class results report ≈ 1.22× mean end-to-end vs vLLM, P4 — vendor/author reported).
- **Verification:** every encode is followed by a decode-and-hash check before the original is released.

8.3 Checkpoints: chunk dedup, delta chains, retention

- **Sharded async save** of model, optimiser and RNG/data-loader state; each shard split into content-defined chunks (~1–4 MiB). Unchanged chunks (frozen embeddings, rarely-updated experts, exactly-repeated optimiser zeros) dedup automatically.
- **Delta chain for consecutive checkpoints:** for every *N*-th checkpoint store a full snapshot (keyframe); in between store XOR/byte-diffs of tensor payloads, compressed (zstd with trained dictionary). Late in training, parameter changes are small, so consecutive BF16 states share high-order bytes and compress well. Keyframes bound restore cost to ≤ *N* delta applications.
- **Optimiser state:** kept in the precision the training recipe uses (e.g., BF16 moments where validated); excluded from release artifacts; dropped from cold tier except at keyframes.
- **Restore test:** automatic periodic resume-from-checkpoint test on a tiny model to catch silent corruption.

8.4 Training data: shards, manifests, dedup

Object	Format and process
Tokenised shards	Fixed-size packed token arrays (uint16 if vocab ≤ 65k else uint32) in ≈ 256 MiB shards, memory-mapped; document boundaries in a sidecar offsets array; per-shard stats (tokens, domain mix, language, dedup status). Packing avoids padding waste; sequence-parallel loaders slice by offset.
Dataset manifest	Ordered list of shard hashes, mixture weights/curricula, tokenizer hash, filtering version, licence tags. Training reads *only* manifests: data order is a pure function of (manifest, seed, step), which makes resume exact.
Dedup & contamination	Exact (hash) and near-duplicate (MinHash/LSH-class) detection at ingest; n-gram overlap scan against every eval suite to flag contamination; results stored in the manifest so evaluations can be re-interpreted later.
Calibration sets	Small, named, versioned sets (general, domain-mixed, generative-sample, long-context); calibration sensitivity is a documented risk (P1 F9), so the set hash is part of every PTQ recipe hash. Calibration statistics (activation Hessians $H = X^T X$, channel scales, router stats) are cached per (model hash, set hash, layer).
Hessian/statistics storage	Per-layer $d \times d$ matrices are large; store in FP32 only until used, then keep a compressed form (eigendecomposition truncated to energy threshold, or diagonal + low-rank) with reconstruction-error metadata. Recompute on demand is cheaper than storing for most layers.

8.5 Teacher and student data

Teacher-output cache: why top-k

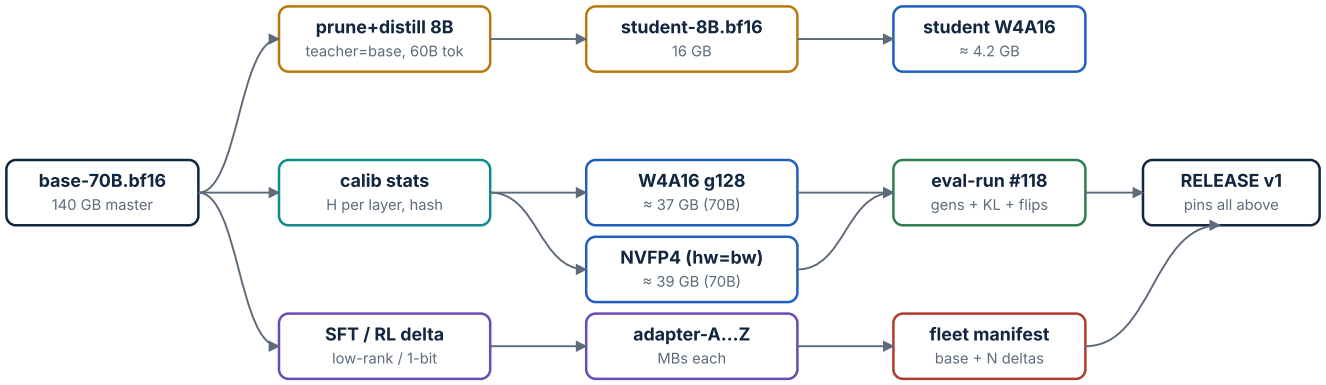
Offline logit distillation needs the teacher's distribution at each position. Storing full-vocabulary logits is hopeless; storing the top-k probabilities captures almost all of the mass. DERIVED for a 128k-vocabulary teacher and 1 B training tokens:

Storage scheme	Bytes per token	Total for 1 B tokens
Full logits, FP16	131,072 × 2 = 262,144	≈ 262 TB
Top-64 (4-byte index + 2-byte prob)	64 × 6 = 384	≈ 384 GB (**≈ 680× smaller**)
Top-32 with 8-bit log-prob + varint-delta indices (≈ 2.4 B per entry)	≈ 77	≈ 77 GB (≈ 3,400× smaller)
Sampled-token + teacher log-prob only (sequence-level KD)	≈ 6	≈ 6 GB

- **Tail handling:** store the residual mass (1 − Σ top-k) as one scalar and distribute it uniformly or by the student's own tail at train time; record top-k coverage statistics so too-small k is detectable.
- **Cache key:** (teacher artifact hash, tokenizer hash, prompt hash, sampling params, k, quantisation scheme). Same key ⇒ reuse across students and experiments.
- **On-policy distillation** cannot be cached (student samples change); the teacher runs live. Log only aggregate and sampled diagnostics (per-token KL, teacher-student agreement) — not the full stream.
- **Reasoning traces:** store prompt + trace + final answer + verifier outcome; compress with dictionary-trained zstd; tag teacher identity and licence for the policy gate (TR-8).

Student lineage

Each student records: parent hash, pruning plan hash, distillation data manifest + teacher hash, token budget consumed, evaluation reports. This is the lineage graph shown in Figure 8.2.



Edge = 'derived from, by recipe hash'. Same recipe + same inputs → same artifact hash (determinism is a testable requirement, NFR-R1).

Figure 8.2 Lineage DAG for a 70B project with a pruned/distilled student, quantised variants, fleet deltas and a release that pins them. Illustrative sizes follow the table in §8.2.

8.6 Logs, metrics and evaluation outputs

Stream	Format	Retention and compaction
Training metrics (loss, grad norm, LR, per-layer stats, FP4 diagnostics)	Columnar Parquet, partitioned by run; step-indexed; float16/delta-encoded where precision allows	Raw: full run. Rollups (1-, 10-, 100-step) kept forever. Per-layer histograms stored as compact sketches.
System telemetry (GPU util, bandwidth counters, power, temp)	Columnar, 1 s sampling during runs	Raw 30 days → 10 s rollups 1 year.
Kernel benchmarks	Table keyed by (chip fp, kernel, shape bucket, config) with median/IQR and residual	Kept indefinitely (small); drives autotuner cache and regression CI.
Evaluation generations	JSONL → columnar with dictionary-trained zstd; token ids plus text; logits *not* stored except sampled top-k for KL suites	Kept for every gated artifact; others 90 days. Enables re-scoring without GPUs.

Serving traces	Request-level spans (arrival, queue, prefill, decode, cache hits, speculation stats) plus sampled full prompts under privacy policy	Sampled; PII-scrubbed; short retention.
Event/audit log	Append-only, signed batches: who ran which recipe, which teacher, which gate decision	Permanent; small.

8.7 Retention, promotion and garbage collection

1. **Pin:** anything referenced by a release manifest, a golden dataset or a published claim-ledger entry is pinned and never collected.
2. **Keep:** for each run, keyframe checkpoints, the best-validation checkpoint, the pre-anneal checkpoint and the final checkpoint; all EvalReports and metric rollups.
3. **Demote:** artifacts not on the current Pareto frontier of any active experiment, and not accessed for *D* days, move hot → warm → cold; variants with pinned recipes may be dropped in the cold tier and flagged "regenerable".
4. **Collect:** mark-and-sweep from pins; delta-chain integrity is checked first (never collect a keyframe that live deltas depend on).
5. **Dry-run first:** GC always produces a report (bytes reclaimed, objects affected) that the UI shows before the sweep runs.

8.8 Access paths and performance

- **Loading weights:** mmap + parallel chunk reads; direct-to-device transfer where the driver supports it; target ≥ 70% of device sequential bandwidth (NFR-P1). For fleets, load the base once and hot-swap adapters.
- **Streaming training data:** shard-level prefetch with deterministic shuffling; per-worker readahead sized from measured storage latency.
- **KV tiers share the address space:** KV blocks are hashed by (model hash, prefix hash, block index); a cold prefix on NVMe can be reloaded faster than recomputed once prefill time exceeds transfer time — the scheduler uses the cost model to decide.
- **Query layer:** an embedded analytical engine over Parquet for UI queries; the metadata DB for lineage and manifests.

> **EXPECTED COMPACTION (TO BE MEASURED AT GATE G1/G3)**
> Sources of savings, from least to most uncertain: chunk dedup across checkpoints and variants; entropy-coded cold weights (≈ 30% on BF16); top-k teacher caches (≈ 680× vs full logits); delta-stored fleet roles (> 10× per fine-tune in P4's BitDelta report); columnar telemetry. The KPI of ≥ 3× versus a naïve layout (G7) is a conservative composition of these; the storage explorer (§9) reports the achieved ratio per project.

9 Interfaces: Web App and Desktop App

Two shells over one control-plane API. The mockups in this chapter are **wireframe-level prototypes** showing information architecture, charts and interactions; all values are illustrative placeholder data — they are not measurements and not predictions.

9.1 Options compared

Option	Strengths	Weaknesses	Role in plan
Web app (browser, served by control plane)	Team access, remote clusters, zero install, easy sharing of links to runs and comparisons, centralised metadata DB	Needs a reachable server; no native GPU/OS integration; large local files awkward	**Primary**. Built first (Phase P6).
Desktop app (webview shell + native sidecar)	Local GPU telemetry and micro-benchmarks, offline work, direct file access (drag a checkpoint in), system-tray job status, SQLite single-user mode	Per-OS packaging and updates; duplicate state if not careful	**Second shell** sharing ≈ 90% of UI code (Phase P7).
CLI / TUI	Scripting, CI, SSH-only machines; the API's reference client	Poor for comparing many runs visually	Always present; thin client.
Notebook SDK	Ad-hoc analysis, custom plots, teaching	Not a product surface; state lives in notebooks	Supported via SDK; no dedicated UI.

Design principles for the UI

1. **Provenance on every number** — badge: measured here · reproduced · paper-reported · derived · vendor; hover shows the ledger entry (UI-3).
2. **Uncertainty is visible** — confidence intervals and a "within noise" marker; no sorted leaderboards of indistinguishable values.
3. **Bytes and bounds first** — every config view starts with fit/bound/ceiling, so users see *why* a recipe is plausible before running it.
4. **Compare by default** — any table row can be added to a comparison tray; comparison is one click away from anywhere.
5. **Same data, same URL** — every view is deep-linkable (web) and addressable (desktop).
6. **Accessible and print-friendly** — colour is never the only encoding; light/dark themes; export of any view as PNG/CSV/PDF.

9.2 What the desktop shell adds

- **Native sidecar** (Rust): GPU/accelerator telemetry (utilisation, memory, clocks, temperature, power), micro-benchmark runner with clock locking where permitted, local artifact-store daemon, file-system integration (open/drag model folders, GGUF/safetensors association).
- **Offline mode**: SQLite metadata and embedded query engine; later sync to a team server by replaying content-addressed objects (idempotent by design).
- **Tray/status**: job progress, gate failures, thermal throttling alerts.
- **Local sandboxing**: kernels under test run in a child process with a watchdog so a bad kernel cannot hang the UI.

9.3 Screen inventory

#	Screen	Questions it answers	Shell
M1	Command centre	What is running, what passed its gates, what is the current Pareto frontier, which claims are untested?	Web + desktop
M2	Recipe builder	Will this pipeline fit, how fast can it be at most, and what will it cost?	Web + desktop
M3	Experiment comparison	Which candidates are genuinely better, on which axes, and are differences real?	Web + desktop
M4	Kernel lab	Is this kernel layout efficient on this chip, and where is it leaving performance?	Desktop-first, web (remote)
M5	Training monitor	Is the run healthy, and is the low-precision gap opening?	Web + desktop
M6	Serving simulator	What concurrency/context can this artifact serve; when does speculation stop helping?	Web + desktop
M7	Storage explorer	Where do the bytes go, what is the lineage, what would GC reclaim?	Web + desktop
M8	Fleet topology	How do agents share weights/caches, and where is context growing?	Web
M9	Desktop local-hardware view	What is my local GPU doing right now, and what will it support?	Desktop

9.4 Prototype mockups

M1 — Command centre (web)

https://forge.example/projects/llm-research/overview

◆ Forge

- Overview
- Hardware
- Recipes
- Experiments
- Kernels
- Training
- Serving
- Storage
- Fleets
- Claims

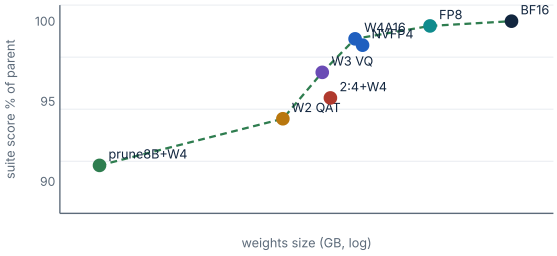
Active runs
6
2 training · 3 eval · 1 bench

Gates passing
14 / 17
3 blocked long-ctx · safety · noise

Storage compaction
3.4×
vs naïve layout · 41 TB → 12 TB

Claims ledger
22 / 61
reproduced · 9 contradicted · 30 untested

Pareto frontier — Llama-class 70B · target chip A illustrative



Claims needing attention

Claim	Src	Status
W4A16 ≈ 4× to B=16	P1	reproduced
EAGLE-3 gain at B=64	P7	partial
RHT helps NVFP4 RTN	P5	contradicted
Shared indexer ~free	P6	untested
REAP 50% experts	P2	untested

Recent runs

Run	Recipe	Status	KL ↓	Flip %	Gate
#118	W4A16-g128 + FP8 KV	done	0.012	2.9	pass
#121	NVFP4 PTQ + QAD	running 62%	—	—	pending
#117	2:4 + W4	done	0.061	9.4	fail: reasoning

M2 — Recipe builder (web)

https://forge.example/recipes/new?model=70B-dense&chip=A

◆ Forge

- Overview
- Hardware
- Recipes
- Experiments
- Kernels
- Training
- Serving

Stage palette

- ▸ Calibrate
- ▸ PTQ (GPTQ/AWQ)
- ▸ Rotate
- ▸ Prune (struct.)
- ▸ Distill
- ▸ QAT / QAD
- ▸ KV quant
- ▸ Pack layout
- ▸ Entropy-code
- ▸ Eval gate
- ▸ Capacity plan

Pipeline canvas planner suggestion · rule P1-Q3



Alts: A FP8 W8A8 · B NVFP4 + QAD (needs FP4 hw) · C baseline arm: 8B student

✓ fits 1×80 GB · weights 36.6 GB · ≈ 262k KV tokens (FP8) · ceiling ≈ 90 tok/s

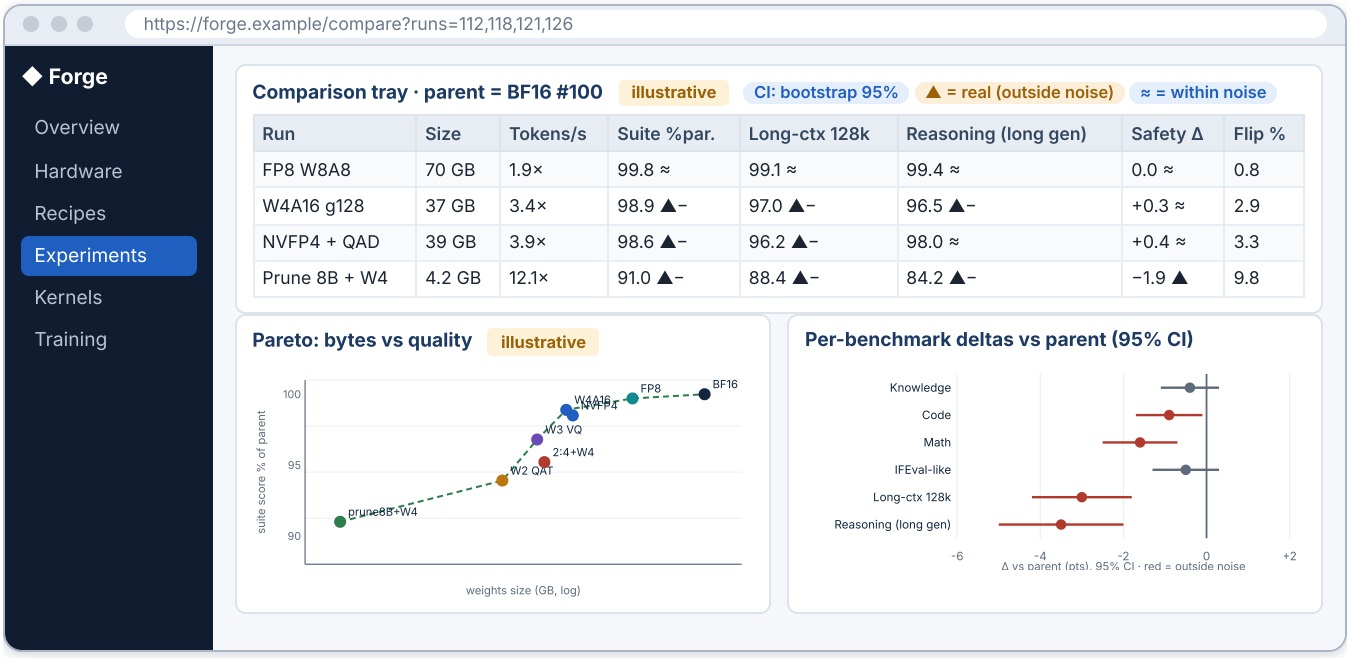
Prediction illustrative

Metric	Pred.
Bound	bandwidth
Tokens/s ceiling	90
Expected (η=.78)	70 ± 6
Quality risk	medium
Eval cost	≈ 3.1 GPU-h
Ledger deps	4 (2 untested)

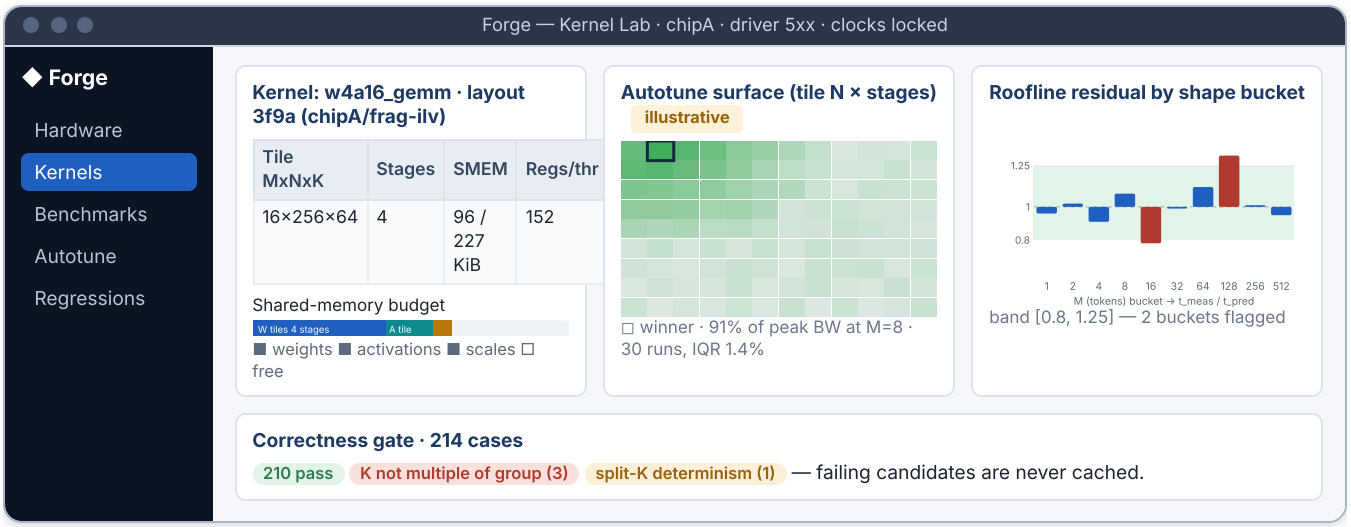
recipe.yaml (live, diffable) — hash 7c1e…a90

```
stages: [calibrate{set: mix-v3}, ptq{method: gptq, bits: 4, group: 128, act_order: true}, pack{layout:
auto(chipA)}, kv{dtype: fp8, residual: 128}, gate{suite: G-W4-v2}]
baselines: [fp8_w8a8, distill_student_8b] · seeds: 3 · stats: bootstrap(2000)
```

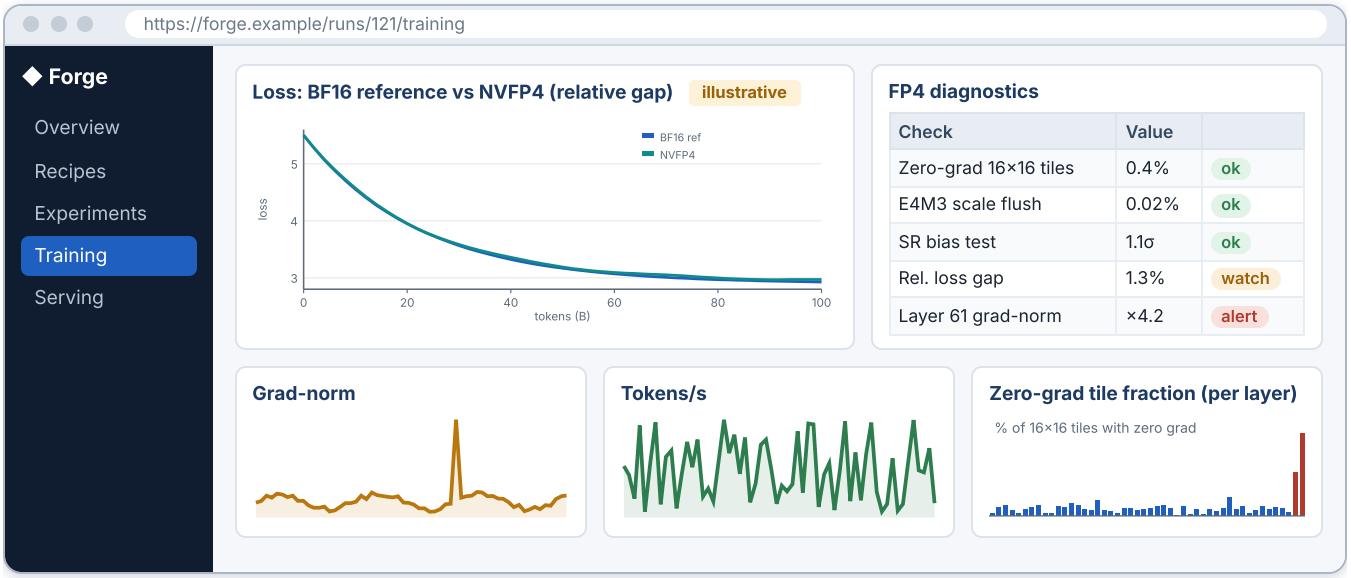
M3 — Experiment comparison (web)



M4 — Kernel lab (desktop-first)



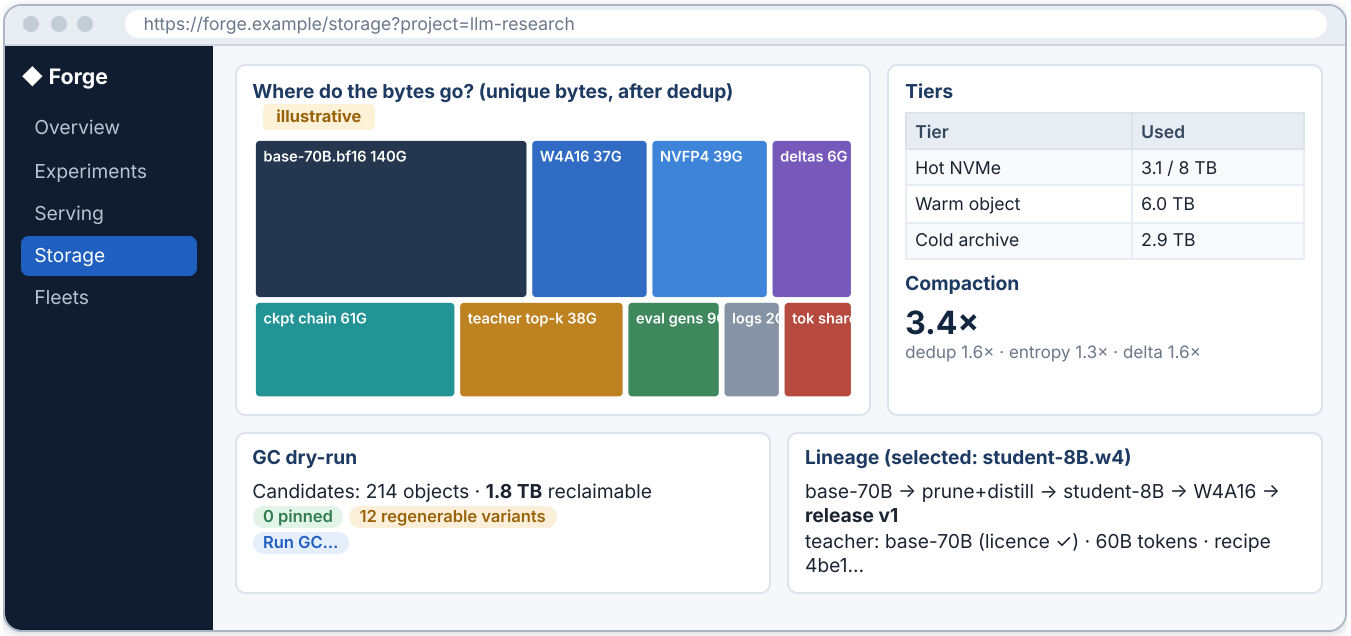
M5 — Training monitor (web / desktop)



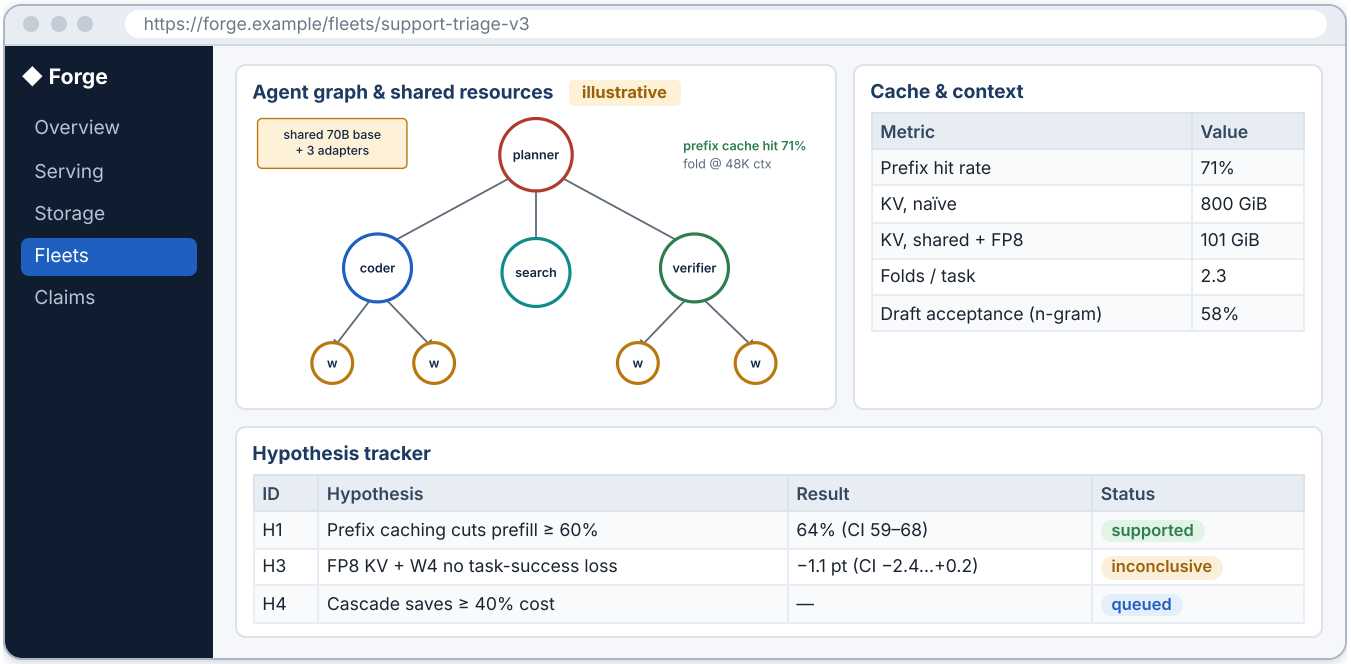
M6 — Serving simulator (web / desktop)



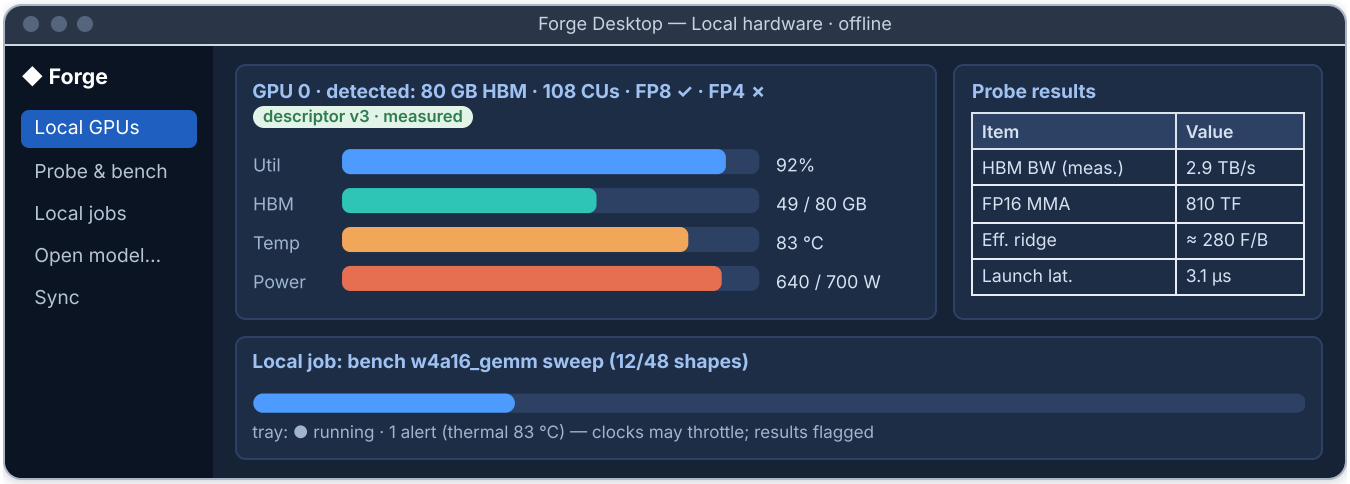
M7 — Storage explorer (web / desktop)



M8 — Fleet topology (web)



M9 — Desktop local-hardware view



10 Evaluation, Validation and Risk

10.1 Evaluation protocol and gates

Four papers (P1, P2, P3, P5) independently conclude that average perplexity is an unreliable guide to deployed behaviour. The evaluation harness is therefore built *first*, versioned like code, and every gate is a **non-inferiority test with a margin** against the parent model rather than a leaderboard position.

Metric definitions

Metric	Definition	What it catches
KL divergence	Mean token-level KL(parent ‖ candidate) on held-out text, teacher-forced; also tail percentile (p99)	Distribution shift invisible to accuracy; cheap, low-variance proxy for the early search rungs
Top-1 flip rate	Fraction of positions where argmax differs from parent (teacher-forced), plus first-divergence position in free generation	Behavioural drift; accumulates over long generation
Calibration (ECE) & entropy shift	Expected calibration error on multiple-choice; mean entropy change	Over/under-confidence introduced by compression
Task suites	Knowledge, code, maths, instruction-following, multilingual, domain tails	Capability loss; use paired comparisons on identical items
Long-generation suite	Multi-thousand-token chain-of-thought, code-then-test, multi-step agentic tasks	Error accumulation (P1 F9); reasoning-model fragility (P1, P5)
Long-context suite	RULER, HELMET, NoLiMa-style, multi-turn reuse (SCBench-style), at the deployment length	Eviction/sparse-attention/KV-quantisation failures that needles hide (P3 F7, P6 §13.2)
Safety & bias probes	Refusal rates, jailbreak resistance, stereotype-reliance scores, paired with parent	Independent safety shifts (P2 F7: +3.5% perplexity vs +83.7% stereotype-reliance in one pruning study)
Throughput & latency	TTFT, inter-token latency, tokens/s at stated concurrency/context; energy per token where available	Whether the compression pays on the real stack (P7: measured speed-ups fall short of bounds)

Statistical rules (EV-5)

- **Noise floor first:** run the parent under different seeds, batch compositions and kernels (inference nondeterminism) to estimate the noise floor per metric; gates must exceed it.
- **Paired bootstrap** over items for task metrics; report 95% CIs; flag "≈ within noise" explicitly. P5's audit found most 12B benchmark differences statistically indistinguishable from zero — Forge's UI must never rank such values.
- **Pre-registration:** sample sizes, seeds, tolerance margins and gate thresholds are fixed in the recipe before the run.
- **Multiple comparisons:** when screening many candidates, control false discovery (e.g., Benjamini–Hochberg) before declaring wins.

Gate classes (initial thresholds are TARGETs, recalibrated at G1)

Gate class	Applies to	Must hold
G-LOSSLESS	Paging, prefix caching, entropy coding, disaggregation, re-pack	Outputs identical (bitwise for storage transforms; within documented floating-point nondeterminism for serving features); memory/latency reported.
G-FP8	FP8 W8A8, FP8 KV	KL p99 ≤ 0.01; flip ≤ 1%; suite non-inferior at δ = 0.5 pt; long-context ≥ 99% of parent.
G-W4	W4A16 / MXFP4 / NVFP4 PTQ, 4-bit KV	Suite ≥ 98% (≥ 30B) / ≥ 96% (< 10B); long-context ≥ 95% at deployment length; long-generation reasoning ≥ 95%; flip ≤ 4%; safety deltas within noise.
G-STRUCT	Pruned / distilled students	Meets size target; suite ≥ declared floor vs parent *and* beats "fresh student" baseline arm at equal footprint; safety/bias deltas reported; licence gate passed.
G-LONG	Eviction, sparse/latent/hybrid attention	Multi-hop and aggregation tasks non-inferior; multi-turn reuse non-inferior; no single-benchmark-only evidence.
G-SPEC	Speculative decoding	Distribution-exactness test passes (empirical vs exact joint on small vocab); speed-up measured at production batch distribution, not only B=1.
G-FP4TRAIN	NVFP4 training runs	Loss gap within pre-registered band vs reference over horizon; TR-2 diagnostics green; downstream suite within noise of reference; healing-phase decision documented.

10.2 Verification of the platform itself

- **Kernel correctness:** oracle comparisons, edge-case suite (§6.7), property-based tests on layouts (permute→unpermute identity), cross-chip agreement tests.
- **Numerical-format conformance:** unit tests that reproduce small tables from the papers (e.g., E2M1 grid values, NVFP4 scale arithmetic, the closed-form zero-fraction formula for low-norm channels reported in P5).
- **Determinism tests:** same recipe + inputs ⇒ same artifact hash for deterministic stages (NFR-R1).
- **Storage tests:** encode→decode→hash round-trips; chaos tests on GC (never collect pinned or chain-referenced objects); restore-from-delta-chain tests.
- **Planner tests:** replay recorded experiment histories; planner top-1 vs exhaustive result agreement (G2 goal).
- **UI tests:** screenshot-based regression for charts; deep-link round-trips; accessibility checks.

10.3 Risk register

ID	Risk	Lik.	Imp.	Mitigation / early signal	Phase
R1	Source-paper claims (vendor or single-source) do not reproduce on our hardware	High	High	Claims ledger; Phase-1 baselines; planner defaults only from reproduced claims. Signal: ledger "contradicted" count rising.	P1

R2	No access to native-FP4 hardware (P5's authors had none either)	Med	High	FP4 numerics studied by fake-quant on any GPU; FP4 kernel performance gated on hardware access; cloud rental for burst benchmarking; do not promise FP4 speed-ups before measured.	P8
R3	Perplexity-passing models fail reasoning, long-context or safety	High	High	Harness built first (Phase 0); gates block promotion; long-generation and safety suites mandatory.	P0
R4	Evaluation noise misleads decisions	High	Med	Noise floor, CIs, pre-registration, FDR control (§10.1).	P0
R5	Kernel work dominates schedule; portability explodes	High	Med	Spec-driven generation; portable Triton-class baseline first; hand-tuned variants only for kernels with ≥ 10% of end-to-end time (Amdahl).	P3
R6	Hardware/format lock-in (recipes tuned to one format, P1)	Med	Med	layout_id separation; re-pack tooling; keep a hardware-neutral master and regenerate variants.	P2–P3
R7	Speculation hurts under load	Med	Med	Controller with γ=0; benchmark with speculation off; production-batch-distribution measurements (P7 §5.8, §19.3).	P4
R8	Sparse/hybrid attention hides multi-hop and aggregation deficits (P3 F7, P6)	Med	High	G-LONG gate; multi-hop evaluation; keep dense fallback; treat vendor claims (e.g., SubQ) as unverified.	P5–P10
R9	Distillation legal/ToS and safety issues (P4 F3)	Med	High	Teacher licence gate; audit trail; paired safety evals; prefer open-licence or self-owned teachers.	P5
R10	Model collapse / diversity loss in synthetic-data loops	Low–Med	Med	Mix real data; diversity metrics across generations; hold-out human-written evals.	P5
R11	FP4 training instability or silent zero-gradient channels (P5)	Med	High	Gated experiment; TR-2 diagnostics; BF16 exemptions; healing phase; keep FP8 as default.	P8
R12	Storage corruption or over-aggressive GC	Low	High	Signed manifests; verify-after-encode; dry-run GC; pins; restore drills.	P0, P11
R13	Planner over-trusts a miscalibrated cost model	Med	Med	Residual alerts; exhaustive-search audits; planner explains rule + ledger entries.	P2
R14	Scope creep (three model forms × many chips)	High	Med	Phase gates; MoE and fleet tracks start only after dense pipeline hits G3; chip tiers (HW-4).	All
R15	Calibration-set leakage or sensitivity	Med	Med	Named calibration sets in recipe hash; contamination scans; ablation across sets (P1 F9).	P2
R16	Quantisation-conditioned attacks / safety regressions (documented in P1)	Low	High	Paired safety evals on every released variant; provenance and signing of artifacts.	P11

11 Roadmap

The roadmap is organised around one dependency idea: **measure → compress → accelerate → serve → train → scale out**. Evaluation and baselines come first because every later decision is judged by them; kernels and serving then convert byte savings into latency; training-heavy and exotic tracks (distillation at scale, FP4 training, MoE, fleets) come after the dense pipeline has proven itself. Durations assume a small team (1 lead + Claude Code as the primary implementer, plus 1–2 engineers for kernels and infrastructure) and at least one modern GPU; they are planning estimates, not commitments.

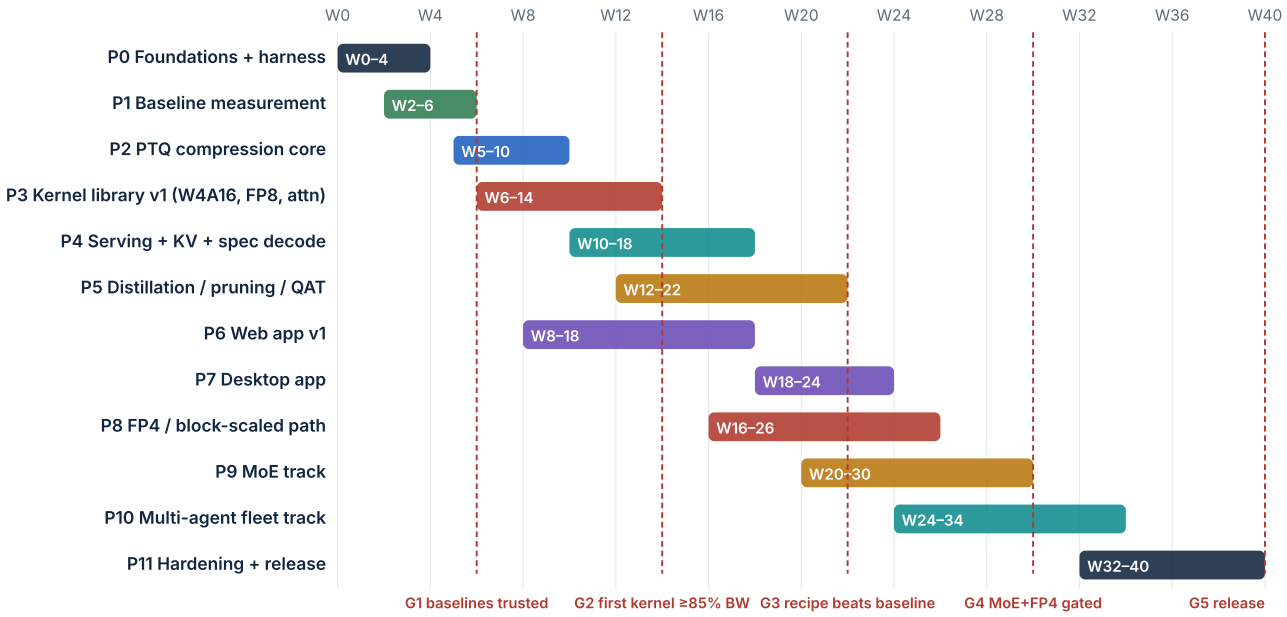


Figure 11.1 Phased plan, ≈ 40 weeks, with five go/no-go gates (dashed). Web-app work (P6) deliberately overlaps with compression and kernels so interfaces are built against real data; the desktop shell (P7) follows once the API has stabilised.

11.1 Phases

Phase	Objective and deliverables	Exit criteria	Why here
P0 Foundations + harness W0–4	Schemas (chip, model, recipe, artifact, eval, claim); artifact store v0 + lineage; determinism utilities; evaluation harness v0→v1 (KL, flip, ECE, suites, long-context, long-generation, safety, statistics); telemetry lake; job runner; API skeleton + CLI.	Parent model evaluated twice → noise floors recorded; artifact round-trip + hash determinism tests green.	Every later number depends on trusted measurement and storage (R3, R4, R12).
P1 Baselines W2–6 → **G1**	Capability probe + micro-benchmarks → chip descriptors; cost model v0; BF16 serving baselines on reference models; claims ledger seeded and first reproductions; fresh-student baseline arms defined.	**G1:** noise floors and baselines trusted; ≥ 10 high-dependency claims have a status; KPI targets recalibrated.	Converts the papers' claims into our evidence (R1).

P2 PTQ compression core W5–10	Calibration registry; RTN/GPTQ/AWQ/SmoothQuant; fake-quant format library (INT, FP8, E2M1+scales); KV quantisation; rotation module; recipe engine with stage caching; first gated W8/W4 artifacts.	FP8 and W4A16 artifacts for the 8B model pass G-FP8 / G-W4 or fail with documented reasons.	Tier-1 production-default techniques (P1 priority ranks 1–2) deliver value earliest.
P3 Kernel library v1 W6–14 → **G2**	Kernel spec + oracles + correctness harness; layout deriver; W4A16 GEMM (portable then tuned); FP8 GEMM integration; paged GQA decode attention with FP8 KV; autotuner + cache; kernel regression CI; cost-model calibration.	**G2:** W4A16 kernel ≥ 85% of measured bandwidth at B ≤ 16 on the primary chip; end-to-end ≥ 2.5× decode vs BF16; planner v0 predictions within ±25%.	Converts byte savings to latency; starts as soon as PTQ produces layouts to pack.
P4 Serving + KV + speculation W10–18	Engine (or adapter) with continuous batching, paged KV, prefix caching, chunked prefill; KV tiers; capacity planner; speculation controller (n-gram first, then EAGLE-style drafter); load tests and capacity curves.	Published capacity curves; speculation never worse than off under the production batch distribution; exactness test green.	Needs P3 kernels; lossless features give risk-free wins (P3 F5).
P5 Distil / prune / QAT W12–22 → **G3**	Teacher top-k cache builder; distillation trainer (offline + on-policy); GQA-aware structured pruning; prune→distil pipeline 8B→≈4B; QAT/QAD; licence gate; optional: low-rank/delta experiments, sparse-attention adaptation.	**G3:** a compressed dense artifact and a distilled student beat their baseline arms at equal footprint; storage compaction measured vs 3× target; planner agrees with exhaustive search ≥ 80%.	Most compute-hungry dense work; benefits from stable harness, store and recipe engine.
P6 Web app v1 W8–18	Design system; M1, M2, M3, M5, M6, M7; provenance badges; CIs; deep links; export; roles.	Any two runs comparable in ≤ 3 clicks; first-baseline-in-an-hour walkthrough passes with a new user.	Starts once API and early data exist; informs P0–P5 outputs.
P7 Desktop app W18–24	Webview shell + native sidecar; local GPU telemetry; M4 and M9; offline mode; packaging/update path.	Offline run + later sync reproduces identical artifact hashes.	After API stabilises; reuses ≈ 90% of web UI.
P8 FP4 / block-scaled path W16–26	Hardware access decision; block-scaled GEMM + scale layouts + fused quantiser; MR-GPTQ-style FP4 PTQ; FP4 training pilot with TR-2 diagnostics.	FP4 vs W4A16-INT4 compared on accuracy *and* speed (P1: not an automatic upgrade); pilot loss-gap within band or FP4 training stays research-only.	Tier-2/3 techniques; needs P2/P3 foundations and special hardware.
P9 MoE track W20–30 → **G4**	Router stats; grouped GEMM/routing kernels; expert prune/merge; MoE quantisation; parallelism planner; quantised all-to-all; speculation behaviour under MoE.	**G4:** MoE reference passes G-W4 / pruning gates with no domain-tail regressions; decision made on FP4 training and MoE defaults.	Needs dense pipeline proven (R14); MoE quantisation young (P1).
P10 Multi-agent fleets W24–34	Fleet manifest/runtime; prompt linter; adapter/delta kernels; context manager; fleet scheduler + traces; M8; agent evaluation; H1–H5 tests.	Each hypothesis resolved as supported / inconclusive / refuted with data.	Builds on P4 caching and P5 students; papers give components, not proof (§7.3).
P11 Hardening + release W32–40 → **G5**	Security review; signing; docs/tutorials; regression and chaos suites; shadow/canary pipeline rehearsal; GC drill; retrospective and roadmap v2.	**G5:** release candidate passes all gates on a clean machine; docs enable an outside user to reproduce a baseline.	Reliability work is cheapest after APIs settle.

11.2 Gates

Gate	Week	Question asked	Evidence required	If it fails
G1	W6	Can we trust our measurements and do the papers' key claims hold here?	Noise floors; reproduced/contradicted statuses for top claims; chip descriptors; recalibrated KPIs	Extend P1; do not start kernel tuning against unverified baselines.
G2	W14	Do our kernels turn byte savings into latency?	≥ 85% bandwidth; end-to-end ≥ 2.5×; planner residuals in band	Reduce scope to vendor/open kernels; keep spec/oracle work.
G3	W22	Does Forge find recipes that beat the baselines?	Compressed and distilled artifacts beat equal-footprint baselines; storage target; planner agreement	Re-examine planner rules; hold P8–P10 starts.
G4	W30	Are FP4 training and MoE ready to become supported defaults?	Diagnostics green; loss-gap bands; tails stable; hardware evidence	Keep as experimental flags; proceed to P10/P11 on dense + MoE-PTQ.
G5	W40	Is the system releasable?	Clean-machine reproduction; security review; docs; regression suite	Extend P11; ship dense-only.

11.3 Resources and assumptions

Resource	Indicative need	Notes
Development GPU	1 × 24–80 GB modern GPU for P0–P4; larger memory strongly preferred for 70B-class tests	Cloud rental acceptable for 70B-class runs.
Multi-GPU node (4–8 GPUs)	P5 distillation/pruning, P9 MoE, interconnect-aware planning	Burst rental is typically enough.
Native FP4 hardware	P8 only, burst access	Dependency R2; plan fake-quant fallback.
Storage	≈ 20–50 TB across tiers for the full plan (dominated by checkpoints and teacher caches before dedup)	Dedup + top-k caches keep this tractable (§8).
Data	Open pretraining-like mix (tens of billions of tokens), calibration sets, evaluation suites with licences recorded	Licence/contamination checks at ingest.
People	Lead/architect; Claude Code for implementation; 1 kernel engineer; 1 infra/UI engineer (part-time)	Kernel engineering is the scarcest skill (R5).
Compute budget	To be set at G1 from measured costs; planner provides GPU-hour estimates per recipe	Budget is a planner input, not an afterthought.

11.4 Working with Claude Code: handoff conventions

You indicated that the coding will be done in Claude Code. To make this document directly usable as a build brief:

1. **Requirement IDs are the unit of traceability.** Each Claude Code task cites the IDs it satisfies (e.g., `KR-2, KR-9`) and the checklist item (`C-043`).

2. **Test-first with oracles.** For each task, the first commit is the failing acceptance test (reference oracle, property test, or gate); the second is the implementation.
3. **Suggested repository layout** (monorepo):

```
forge/
  core/        schemas, hashing, config, errors            (Python)
  store/       content-addressed store daemon + client     (Rust/Go + py bindings)
  hw/          chip descriptors, probes, microbench        (Python + native sidecar)
  kernels/     specs/, ref/ (oracles), portable/, tuned/, autotune/, layouts/
  compress/    calibrate/, ptq/, qat/, prune/, distill/, lowrank/, delta/, lossless/
  train/       precision ladder, trainers, diagnostics
  serve/       engine adapter, kv/, spec/, scheduler/, fleet/
  eval/        metrics/, suites/, stats/, gates/
  planner/     costmodel/, rulebase/, search/, claims/
  api/         control-plane service + OpenAPI
  ui/          web/ (SPA), desktop/ (shell + sidecar)
  docs/        this plan, ADRs, runbooks
```

4. **Architecture Decision Records (ADRs)** for every choice flagged “recommendation” in §4.4 and every deviation from this plan.
5. **Definition of done** (per task): requirement IDs cited; tests + oracles pass; documentation updated; benchmark/regression entry added (if performance-relevant); lineage/provenance metadata emitted; no secrets or licences violated.
6. **Small vertical slices:** prefer a thin end-to-end path (calibrate → quantise → pack → serve → evaluate → report) over deep single-layer work; deepen after the slice runs.

12 Ordered Checklist: What to Do, in What Order

This is the executable form of the roadmap. Items are numbered `C-nnn` (gates `G1–G5`) in dependency order *within* each phase; phases overlap as shown in Figure 11.1. Each item states **why it sits where it does** and a concrete **done-when** condition so that a Claude Code task can be written directly from it. Tick boxes are for printing.

ORDERING RULES USED

1. **Measurement before optimisation** — harness, baselines and noise floors precede every technique.
2. **Lossless and production-default before frontier** — FP8/W4A16/paged KV/prefix caching before FP4 training, sub-3-bit codes or sparse-attention retraining.
3. **Oracle before kernel, kernel before engine** — correctness references exist before any tuned code; engines consume kernels that already pass the gate.
4. **Dense before MoE before fleets** — each later form reuses and stresses the earlier pipeline.
5. **Gate before scale-out** — expensive tracks (P8–P10) start only after G3 shows the pipeline beats baselines.

Phase 0 — Decisions, access and foundations W0–4

- [ ] `C-001` **Inventory hardware and access.** List every chip available (model, memory, interconnect, driver) and cloud budget for burst use.
  Why this order: Every kernel, format and recipe decision is chip-dependent; an unknown inventory silently blocks P3 and P8.
  Done when: Written inventory; one named ‘primary chip’.

- [ ] `C-002` **Choose reference models.** One ≈ 8B dense, one ≈ 70B-class dense (cloud OK), one small MoE, one agent-fleet scenario; record licences and distillation permissions.
  Why this order: Fixed targets make results comparable; the small models iterate fast, the large one exposes capacity effects (Example A).
  Done when: Models downloaded, hashed, licences logged (TR-8).

- [ ] `C-003` **Create repo, CI and Claude Code project instructions.** Monorepo layout (§11.4), environment pinning, pre-commit, CI with CPU tests, a CLAUDE.md that links requirement IDs to this document.
  Why this order: Conventions set before code exists prevent drift across many small tasks.
  Done when: CI green on an empty skeleton; conventions documented.

- [ ] `C-010` **Define schemas.** Chip descriptor, model config, recipe, artifact manifest, EvalReport, Claim, Deployment (Appendices B–C) with validation tests.
  Why this order: Schemas are the contracts between all layers; changing them late is the most expensive refactor.
  Done when: Schemas versioned; golden-file tests pass.

- [ ] C-011 **Artifact store v0.** BLAKE3 content addressing, chunking, manifests, SQLite lineage DB, CLI put/get/verify.
  Why this order: Reproducibility and compactness are properties of storage (DT-1); every other component writes through it.
  Done when: Round-trip + hash-determinism tests; dedup demo (two near-identical checkpoints).
- [ ] C-012 **Determinism and environment capture.** Seeds, library/driver/kernel versions and hardware fingerprint recorded in every run.
  Why this order: Needed to interpret noise (R4) and to reproduce (NFR-R1).
  Done when: Re-run of a stage yields identical hash or a documented diff.
- [ ] C-013 **Evaluation harness v0.** KL, flip rate, ECE, perplexity, three task suites, bootstrap CIs, noise-floor estimator.
  Why this order: Built before any optimisation: perplexity alone is not an acceptance test (P1 F9, P2 F7).
  Done when: Parent evaluated twice; noise floors stored.
- [ ] C-014 **Evaluation harness v1.** Long-generation suite, long-context suite at deployment length, safety/bias probes, contamination scan.
  Why this order: These suites find the failures that compression causes first (R3, R8).
  Done when: Suites run on parent; known-bad model (heavily pruned) fails as expected.
- [ ] C-015 **Telemetry lake v0 and job runner.** Parquet metrics, embedded query, local + SSH job runner with log capture.
  Why this order: Needed by P1 measurements and all UIs.
  Done when: A job streams metrics into the lake and is queryable.
- [ ] C-017 **Control-plane API skeleton and CLI.** OpenAPI, auth stub, resources from §4.3, thin CLI client.
  Why this order: One API serves every shell (UI-1); CLI is the reference client.
  Done when: CLI can register a model, start a job, read an EvalReport.

Phase 1 — Baselines and hardware profiling

W2–6 → Gate G1

- [ ] C-020 **Capability probe and micro-benchmarks.** Auto-fill chip descriptor; measure HBM bandwidth, MMA rates per dtype, shared-memory bandwidth, launch latency, dequant ALU throughput.
  Why this order: Measured numbers replace datasheet numbers (HW-2); effective ridge is below peak in real stacks (P7).
  Done when: Descriptor for each chip, versioned, with measurement provenance.
- [ ] C-021 **Cost model v0.** Byte accounting, KV sizing, roofline time, batch crossover, quantiser overhead (§5.1).
  Why this order: Lets the planner reject infeasible recipes before spending GPU time.
  Done when: Reproduces Examples A and B and P1's $B \approx 74$ / 295 crossover to within rounding.
- [ ] C-022 **BF16 serving baselines.** Tokens/s and latency vs batch × context for each reference model with an existing open engine.
  Why this order: Every speed-up claim needs a same-hardware baseline (R1).
  Done when: Baseline tables stored; CI of repeated runs known.
- [ ] C-023 **Seed the claims ledger.** Top ≈ 30 claims from P1–P7 ranked by number of dependent rules; reproduce the first few (W4A16 vs batch, FP8 parity, KV size formula).
  Why this order: Prioritises scarce experiment time and prevents building on false premises (principle 12).
  Done when: Ledger entries with statuses; at least 10 resolved.

- [ ] C-024 **Define baseline arms.** ‘Fresh smaller student’ and ‘next-lower-precision’ arms for each reference model.
  Why this order: A compression method must beat the moving baseline (P4 F8).
  Done when: Arms specified as recipes with token budgets.

- [ ] G1 **GATE G1 — baselines trusted.** Review noise floors, descriptors, ledger statuses; recalibrate KPI targets (§2.3).
  Why this order: Everything downstream is judged against these baselines.
  Done when: Go/no-go recorded.

Phase 2 — PTQ compression core W5–10

- [ ] C-030 **Calibration registry and statistics cache.** Named calibration sets; cache Hessians/scales per (model, set, layer).
  Why this order: Calibration sensitivity is a documented risk (R15); caching makes sweeps cheap.
  Done when: Cache hit on repeated recipe.

- [ ] C-031 **RTN + group quantisation and reference dequant.** INT4/INT8 symmetric/asymmetric, group 32–128; reference dequant used as oracle later.
  Why this order: Baseline method and the correctness oracle for kernels.
  Done when: Round-trip error matches theory (≈ 6 dB/bit) on synthetic data.

- [ ] C-032 **GPTQ-family, AWQ-style, SmoothQuant-style W8A8.** Layer-wise Hessian reconstruction; activation-aware scaling; smoothing.
  Why this order: Tier-1 production defaults (P1 ranks 1–2).
  Done when: W4A16 and W8A8 artifacts for the 8B model with eval reports.

- [ ] C-033 **Fake-quant format library.** INT, FP8 (E4M3/E5M2), E2M1 + E4M3/E8M0 block scales; stochastic rounding; unit tests against the papers’ tables (E2M1 grid, scale arithmetic).
  Why this order: Shared numerics for PTQ, QAT and kernel oracles; validates our reading of P5.
  Done when: Conformance tests green; NVFP4 vs MXFP4 MSE ordering reproduced.

- [ ] C-034 **KV-cache quantisation (reference).** FP8 KV and KIVI-style per-channel-key / per-token-value with residual window in reference attention.
  Why this order: KV is the second memory stream (P1 §2.2.3, P3 F1).
  Done when: Eval at full target context; accuracy vs bytes plotted.

- [ ] C-035 **Recipe engine.** Declarative stages, hashing, per-stage caching, YAML loader, CLI run, lineage emission.
  Why this order: Makes every experiment reproducible and cheap to repeat (CP-12).
  Done when: Re-running a cached recipe is instant; changing one stage reruns only downstream.

- [ ] C-036 **First gated artifacts.** FP8 and W4A16 for the 8B model through G-FP8 / G-W4; compare in CLI.
  Why this order: Proves the end-to-end slice before deepening any layer.
  Done when: Two EvalReports with verdicts.

- [ ] C-037 **Rotation module.** Randomised Hadamard / learned rotation; evaluate with and without per format, noting P5’s finding that RHT does not help NVFP4 RTN.
  Why this order: Avoids cargo-culting rotations; rotation helps W4A4 on INT hardware (P1 F4).
  Done when: Per-format table of when rotation helps.

Phase 3 — Kernel library v1 W6–14 → Gate G2

- [ ] C-040 **Kernel spec, oracles and correctness harness.** Spec format (Appendix C.2); reference implementations; tolerance rules; edge-case suite (§6.7).
  Why this order: Correctness before speed; failing candidates must never be cached (KR-1, KR-9).
  Done when: Harness rejects a deliberately wrong kernel.
- [ ] C-041 **Layout deriver v0 and W4 weight packer.** Implement §6.3 for the primary chip; offline permute/interleave/pack with unpermute tests.
  Why this order: Layout is part of the artifact (layout_id), so it must exist before kernels.
  Done when: Property test: pack→unpack identity for random shapes.
- [ ] C-042 **Portable W4A16 GEMM.** Triton-class kernel validated against oracle; benchmark across shapes.
  Why this order: Coverage for unknown chips and a baseline for the tuned kernel (NFR-V1).
  Done when: Passes harness; speed-up vs BF16 at B=1–16 recorded.
- [ ] C-043 **Tuned W4A16 GEMM + autotuner.** Chip-specific schedule (async rings, fast dequant, striped partition, L2/smem-resident activations); autotune cache.
  Why this order: Main lever for decode (P1 §14.2).
  Done when: ≥ 85% bandwidth at B ≤ 16, ≥ 3.2× vs BF16 GEMM (TARGET).
- [ ] C-044 **FP8 GEMM and fused activation quantisation.** Wrap vendor FP8 GEMM; fuse norm→quantise; scaling granularities.
  Why this order: Production default on FP8 hardware (P1 Q2).
  Done when: ≥ 90% of vendor GEMM; G-FP8 passes end to end.
- [ ] C-045 **Paged decode attention.** GQA grouping, split-KV, FP8 KV dequant, reference + portable + tuned; MLA variant when reference arch needs it.
  Why this order: Attention dominates at long context; arithmetic intensity is only g FLOP/B (P3 F1).
  Done when: ≥ 85% bandwidth at 32K–128K; matches reference attention.
- [ ] C-046 **Kernel benchmark DB and regression CI.** Store medians/IQR per (chip, kernel, shape, config); alert on >5% drops.
  Why this order: Keeps performance from rotting (KR-10).
  Done when: CI fails on injected slowdown.
- [ ] C-047 **Calibrate the cost model.** Fit η factors from kernel data; residual map.
  Why this order: Planner credibility (R13).
  Done when: Predictions within ±25% on held-out shapes.
- [ ] G2 **GATE G2 — kernels pay off.** Review kernel KPIs and end-to-end decode speed-up.
  Why this order: Decides whether to continue custom kernels or lean on vendor/open ones.
  Done when: Go/no-go recorded.

Phase 4 — Serving, KV management and speculation W10–18

- [ ] C-050 **Serving engine or adapter.** Continuous batching, paged KV, radix prefix cache, chunked prefill (SV-1); decide build vs adapt via ADR.
  Why this order: Lossless features are the safest wins (P3 F5).
  Done when: Outputs identical with features on/off (within FP nondeterminism).

- [ ] C-051 **Capacity planner.** Implements Example A/B logic with engine overheads measured from C-050.
  Why this order: SV-5; a planner promise must match the live engine.
  Done when: Predicted vs measured max concurrency within 10%.
- [ ] C-052 **KV block manager with tiers.** HBM → host → NVMe; hash-addressed blocks; demote/promote; stats.
  Why this order: Long-context and fleet capacity (SV-2).
  Done when: Prefix reload from NVMe beats recompute above modelled break-even.
- [ ] C-053 **Speculation: n-gram, controller, drafter.** N-gram lookup first; load-aware γ controller with γ=0; EAGLE-style drafter integration; exactness test.
  Why this order: Regime-dependent gains (P7 §5.8); n-gram first for context-copying work (P7 §19.3).
  Done when: G-SPEC passes; never slower than off under production batch mix.
- [ ] C-054 **Load tests and capacity curves.** Tokens/s vs concurrency × context for each artifact; TTFT/ITL tails.
  Why this order: Published curves are the deliverable for deployments (G6).
  Done when: Curves stored and visible in the UI.
- [ ] C-055 **Disaggregated prefill/decode (experiment).** Separate pools with KV transfer.
  Why this order: Tests P3 F5 on our workloads; needed for fleets later.
  Done when: Decision recorded with numbers.

Phase 5 — Distillation, pruning and QAT

W12–22 → Gate G3

- [ ] C-060 **Teacher top-k cache builder.** Generate, store, verify top-k caches with residual mass and coverage stats (§8.5).
  Why this order: ≈ 680× smaller than full logits; reuse across students.
  Done when: Cache keyed and reused; coverage ≥ target.
- [ ] C-061 **Distillation trainer.** Offline logit, sequence-level, on-policy modes; KL variants; fresh-student baseline arm.
  Why this order: Distillation is the default route to smaller models (P4 F1).
  Done when: Student improves over SFT-only baseline on a pilot.
- [ ] C-062 **Structured pruning.** Importance estimation; GQA-aware width pruning; head/depth options; expert prune hooks for P9.
  Why this order: Most reliable route to a smaller dense model (P2 §1.2).
  Done when: Pruned model loads, runs, and has expected shapes.
- [ ] C-063 **Prune → distil run.** 8B → ≈ 4B with tens of B tokens; G-STRUCT evaluation against fresh-student arm.
  Why this order: Core of Gate G3; tests principle 6 and 7 on our data.
  Done when: Report with CIs and verdict.
- [ ] C-064 **QAT / QAD.** Fake-quant training matching the deployed kernel bit-for-bit; QAD from BF16 teacher for reasoning models.
  Why this order: Closes sub-4-bit gaps and fixes reasoning loss (P1 F5, P5 §19.1).
  Done when: QAD artifact passes G-W4 where PTQ failed (or documented why not).

- [ ] C-065 **Teacher licence gate and audit trail.** Block jobs lacking permission; record teacher identity in lineage.
  Why this order: R9 / TR-8.
  Done when: Gate demonstrably blocks a disallowed teacher.
- [ ] C-066 **Optional: low-rank, delta, sparse-attention experiments.** Whitened SVD with healing; BitDelta-style deltas; shared-indexer sparse-attention adaptation vs same-budget dense baseline.
  Why this order: Each has niche or single-source evidence (P4, P6); test, don't assume.
  Done when: Results in ledger as supported/refuted.
- [ ] G3 **GATE G3 — recipes beat baselines.** Review artifacts, storage compaction, planner agreement.
  Why this order: Decides whether to scale out to FP4, MoE and fleets.
  Done when: Go/no-go recorded.

Phase 6 — Web application (parallel from W8) W8–18

- [ ] C-070 **Front-end scaffold and design system.** SPA, typed API client, theming, canvas charting for large series.
  Why this order: Built against real API data early (UI-1).
  Done when: Hello-world page lists runs from the API.
- [ ] C-071 **Core screens.** M1 Command centre, M3 Comparison, M2 Recipe builder, M5 Training monitor, M6 Serving simulator, M7 Storage explorer.
  Why this order: Priorities follow the analysis loop: see → compare → plan → monitor → deploy → store.
  Done when: Each screen has a recorded walkthrough.
- [ ] C-072 **Provenance badges and uncertainty display.** Badges from claims ledger; CIs and 'within noise' markers.
  Why this order: UI-3; prevents false precision.
  Done when: No ranking of indistinguishable values in any table.
- [ ] C-073 **Deep links, export, roles.** Shareable URLs; CSV/PNG/PDF export; read/write roles.
  Why this order: Team use.
  Done when: Link round-trip tests.

Phase 7 — Desktop application W18–24

- [ ] C-080 **Desktop shell and native sidecar.** Webview shell, Rust sidecar (GPU telemetry, probe runner, file integration), local store daemon.
  Why this order: Native access the browser lacks (§9.2).
  Done when: App opens a local model and runs the probe.
- [ ] C-081 **Kernel lab (M4) and local-hardware view (M9).** Layout inspector, autotune surface, roofline residuals, live GPU stats, tray alerts.
  Why this order: Kernel work is most productive next to the hardware.
  Done when: Autotune run visualised live.
- [ ] C-082 **Offline mode and sync.** SQLite + embedded query; replay of content-addressed objects to server.
  Why this order: Idempotent sync by construction.
  Done when: Offline artifact reproduces identical hash after sync.

Phase 8 — Block-scaled FP4 and low-precision training W16–26

- [ ] C-090 **Confirm FP4 hardware access.** Probe for native block-scaled formats; arrange burst access if absent.
  Why this order: R2: do not promise FP4 speed-ups without measurement.
  Done when: Decision: native path vs fake-quant only.
- [ ] C-091 **Block-scaled GEMM, scale layouts, fused quantiser.** NVFP4 and MXFP4 variants; padded/swizzled scale tensors; TN handling.
  Why this order: Kernel K2; fusion is mandatory (P5 Table 11.2).
  Done when: ≥ 2.2× vs FP8 on compute-bound shapes where hardware permits.
- [ ] C-092 **Format-aware FP4 PTQ.** MR-GPTQ-style; compare with INT4 W4A16 on accuracy and speed.
  Why this order: P1: FP4 is not an automatic upgrade over INT4.
  Done when: Per-model recommendation table.
- [ ] C-093 **FP4 training pilot with diagnostics.** Small model; BF16 exemptions; SR; optional RHT/2D scaling; healing phase; TR-2 dashboard.
  Why this order: Gated experiment (R11); evidence base is thin (P5).
  Done when: Loss-gap within band or FP4 training remains research-only.

Phase 9 — Mixture-of-Experts track W20–30 → Gate G4

- [ ] C-100 **MoE profiling.** Router statistics collector; expert usage/saliency; domain tails.
  Why this order: Calibration must include rare experts (P2 F5).
  Done when: Stats for the MoE reference model stored.
- [ ] C-101 **Routing and grouped-GEMM kernels.** Fused top-k, permutation, grouped GEMM in K1/K2/K3 layouts.
  Why this order: KR-7; MoE bandwidth/latency is dominated by these.
  Done when: Matches oracle; ≥ target bandwidth on skinny experts.
- [ ] C-102 **Expert prune/merge and MoE quantisation.** REAP-style saliency; merge; per-expert bit allocation; tail evaluation.
  Why this order: Fastest-moving area (P2 F5) with young quantisation evidence (P1).
  Done when: 25–50% expert reduction within gate or documented limit.
- [ ] C-103 **Parallelism planner and quantised all-to-all.** Enumerate TP/EP/DP/PP; communication byte model; FP8 payloads where link-bound.
  Why this order: Planner support for MoE form (§7.2).
  Done when: Predicted vs measured step time within ±20%.
- [ ] C-104 **Speculation under MoE.** Measure expert-touch growth with γ; controller update.
  Why this order: P7 §5.8: dependence can invert.
  Done when: Controller behaviour validated.
- [ ] G4 **GATE G4 — FP4 training and MoE defaults.** Decide which stay experimental.
  Why this order: Prevents promoting thinly-evidenced features.
  Done when: Go/no-go recorded.

Phase 10 — Multi-agent fleets W24–34

- [ ] C-110 **Fleet manifest, runtime and prompt linter.** Base + role adapters + policies; canonical prompt ordering checker.
  Why this order: Cache-friendliness is designed in (§7.3).
  Done when: Fleet loads, runs two roles on one base.
- [ ] C-111 **Adapter/delta kernels and delta compression.** Segmented-batched adapter kernel; BitDelta-style deltas.
  Why this order: Memory scales with deltas not models (P4).
  Done when: Mixed-role batch matches per-role reference.
- [ ] C-112 **Context manager.** Budgets, folding/summarising triggers, typed eviction policies.
  Why this order: P3 F6: agent policy over raw eviction.
  Done when: Per-agent KV bounded; task success within noise (H5).
- [ ] C-113 **Fleet scheduler, traces and M8.** Critical-path priority; preemption with KV swap; trace capture; topology view.
  Why this order: Attribution of failures to compression vs orchestration.
  Done when: Trace shows per-step tokens/cache hits.
- [ ] C-114 **Agent evaluation and hypotheses H1–H5.** Multi-step tool-use tasks; measure H1–H5 with CIs.
  Why this order: Papers do not evaluate fleets; evidence must be ours.
  Done when: Each hypothesis resolved with data.

Phase 11 — Hardening and release W32–40 → Gate G5

- [ ] C-120 **Security review and signing.** Signed manifests, least-privilege credentials, secret scanning, audit log.
  Why this order: NFR-S1; compression can alter safety behaviour (R16).
  Done when: Review checklist passed.
- [ ] C-121 **Regression and chaos suites.** Kernel/perf regressions, storage chaos (GC, corruption, restore), planner replay.
  Why this order: R12; keeps long-lived artifacts safe.
  Done when: Suites in CI.
- [ ] C-122 **Docs, tutorials, example projects.** Runbooks, ADR index, ‘first baseline in an hour’ guide.
  Why this order: Usability goal G8.
  Done when: Outside user reproduces a baseline.
- [ ] C-123 **Release rehearsal.** Shadow → canary → ramp → rollback on a candidate; GC dry-run and drill.
  Why this order: Validates §7.5 process before it matters.
  Done when: Rollback exercised successfully.
- [ ] C-124 **Retrospective and roadmap v2.** Update ledger, KPIs, risks; plan next-generation items (e.g., diffusion drafters, SSD, new formats).
  Why this order: Field moves quickly (papers cover Oct 2025–Oct 2026).
  Done when: Roadmap v2 approved.

- [ ] G5 **GATE G5 — release.** Final review.
  Why this order: Final item — all prior work feeds it.
  Done when: Go/no-go recorded.

Appendix A — Traceability: Findings → Requirements → Work

Paper finding (short)	Requirement IDs	Phase / checklist items
P1 F1–F2: decode is memory-bound; W4A16 mature, 8-bit solved	KR-2, KR-3, CP-1, SV-5	P2 C-031–036; P3 C-042–044; P4 C-051
P1 F3–F4: second-order PTQ; outliers dictate activation quantisation; rotations	CP-1, CP-3	C-032, C-037
P1 F5: below 4 bits, codebooks and training win	CP-11, TR-5, KR-2 (LUT variants)	C-064; optional sub-3-bit track (not in v1 critical path)
P1 F6–F7: FP4 not automatic upgrade; 8/4-bit training credible	CP-2, KR-3, TR-1, TR-2	P8 C-090–093
P1 F8: KV needs its own recipe	CP-4, KR-4	C-034, C-045
P1 F9 / P2 F7 / P3 F7: perplexity insufficient; evaluation binds	EV-1…EV-7	C-013, C-014
P1 F10 / P5: formats and kernels hardware-specific	HW-1…HW-4, KR-1, KR-9, NFR-V1	C-020, C-040, C-041
P2 F1–F2: 50% one-shot easy, GPU speed-ups rare	CP-5 (warnings)	Planner rules (§5.2); C-062
P2 F3–F4: prune + distil dominant; depth pruning costs generation	CP-5, TR-4	C-062, C-063
P2 F5: MoE pruning and activation sparsity fastest-moving	CP-6, CP-7, KR-7, KR-8	P9 C-100–103; optional TEAL in C-066
P3 F1–F3: KV sets long-context cost; architecture; trainable sparse attention	KR-4, KR-5, TR-6, SV-5	C-045, C-066
P3 F4: post-hoc eviction fails on hard tasks	EV-3, EV-6; planner caution	G-LONG; §5.2 rules
P3 F5: serving layer is the stable high-return layer	SV-1, SV-2	C-050–052, C-055
P3 F6: context management becomes policy	SV-7	C-112
P4 F1–F3: on-policy distillation; reasoning distillation; legal/safety surface	TR-3, TR-8	C-061, C-065
P4 F4–F6: whitened SVD; low-rank rarely wins wall-clock; sharing = compute bill	CP-8, KR-9 (K9)	C-066 (optional)
P4 F7: lossless entropy coding bounded (~30% BF16)	CP-10, DT-2	§8.2; storage tasks within C-011 extensions
P4 F8: baseline moves (smaller over-trained models)	Baseline arms (§5.3)	C-024

P5: scale format drives NVFP4 advantage; SR bias; RHT caveats; zero-gradient tiles; noise audit	KR-3, TR-1, TR-2, EV-5	C-033, C-091, C-093, C-013
P6: selector scan dominates; sharing indexers; parity thinly evidenced beyond 106	KR-5, TR-6, EV-3	C-066; G-LONG
P7: speculation regime-dependent; engines bear overhead	SV-3, SV-4, KR-6	C-053, C-104

Appendix B — Chip Descriptor Schema (sketch)

```
# chip_descriptor.yaml — all fields measured by the probe where possible; "src" records provenance
id: chipA-r3                    fingerprint: sha256:…
vendor: …    family: hopper_class | blackwell_class | cdna3_class | cpu_avx | systolic | unknown
compute_units: 108              clock_mhz_locked: 1755
memory:
  hbm_gb: 80   hbm_bw_gbs: {datasheet: 3350, measured: 2900, src: microbench/stream_copy@r3}
  l2_mb: 50    transaction_bytes: 128    max_vector_load_bytes: 16
on_chip:
  smem_per_cu_kib: 227   smem_banks: 32   bank_width_bytes: 4
  regs_per_cu: 65536     max_regs_per_thread: 255
  async_copy: [cp_async, bulk_async]     cluster_launch: true
matrix_units:
  - dtype: fp16|bf16   atom: [16,8,16]    peak_tflops: {datasheet: 989, measured: 810}
  - dtype: fp8_e4m3    atom: [64,N,32]    peak_tflops: {…}     block_scaled: false
  - dtype: int8        …
  - dtype: fp4_e2m1    block_scaled: {block: 16, scale: e4m3, scale_atom: [128,4]}   # only if probe confirms
sparsity: {two_four: true}
alu: {simple_ops_per_s_t: 60}    launch_latency_us: 3.1
interconnect: {intra_gbs: 450, inter_gbs: 50, topology: switch}
derived: {ridge_flop_per_byte: {fp16: 280, fp8: 560}}   power_limit_w: 700
```

Appendix C — Configuration Examples

C.1 Recipe (YAML)

```
recipe: w4a16-fp8kv-70b
inputs: {model: sha256:base70b_bf16, calib: sha256:mix_v3, chip: chipA-r3}
stages:
  - calibrate: {set: mix_v3, n_seqs: 512, seq_len: 2048, stats: [hessian, channel_scale]}
  - ptq:       {method: gptq, bits: 4, group: 128, act_order: true, damp: 0.01, sym: false}
  - pack:      {layout: auto, target: chipA-r3}                  # emits layout_id
  - kv:        {dtype: fp8_e4m3, residual_window: 128}
  - gate:      {suite: G-W4-v2, parent: sha256:base70b_bf16, seeds: 3, stats: bootstrap_2000}
baselines: [fp8_w8a8, fresh_student_8b]
budget: {gpu_hours: 40, max_wall_hours: 12}
policy: {licence_check: true}
```

C.2 Kernel spec (YAML)

```
kernel: w4a16_gemm
op: "Y[M,N] = X[M,K] @ dequant(Wq[N,K])^T"
dtypes: {x: bf16, w: int4, scale: bf16, acc: fp32, y: bf16}
quant: {group: [64, 128], zero_point: optional, scale_apply: epilogue_if_per_column}
shape_hints: {M: [1, 2, 4, 8, 16, 32, 64, 128], N: model_native, K: model_native}
search:
  tile_m: [16, 32, 64]  tile_n: [128, 256]  tile_k: [64, 128]
  stages: [2, 3, 4, 5, 6]   split_k: [1, 2, 4, auto]
fusion: {prologue: none, epilogue: [bias, residual, act_quant(optional)]}
layout: {weights: fragment_interleaved(hw), scales: col_contig, align_bytes: 16}
oracle: ref.w4a16_gemm_fp32     tolerance: {rel: 2e-3, abs: 1e-3, bitexact_int_paths: true}
kpi: {bw_fraction_min: 0.85, speedup_vs_bf16_min: 3.2, at_M: [1, 16]}
```

C.3 Evaluation gate (YAML)

```
gate: G-W4-v2
parent: required
noninferiority: {margin_pts: 2.0, alpha: 0.05, fdr: benjamini_hochberg}
metrics:
  kl_p99:            {max: 0.05}
  top1_flip:         {max: 0.04}
  suite_score_ratio: {min: 0.98, applies_if: params_b >= 30, else_min: 0.96}
  longctx_ratio:     {min: 0.95, at_length: deployment}
  longgen_reasoning: {min: 0.95}
  safety_delta:      {within_noise: true}
noise_floor: from_parent_repeats(n=5)
verdict: pass | fail | inconclusive(with reasons)
```

C.4 Fleet manifest (YAML)

```
fleet: support-triage-v3
base: sha256:student8b_w4a16
roles:
  planner:  {adapter: sha256:lora_planner,  prompt: p_planner.md,  budget_tokens: 32768, fold_at: 24576}
  coder:    {adapter: sha256:lora_coder,    prompt: p_coder.md,    budget_tokens: 24576}
  verifier: {delta:   sha256:bitdelta_ver,  prompt: p_verifier.md, budget_tokens: 16384}
shared_prefix: [global_system.md, tool_schemas.json]           # must come first; linted
cascade: {routine: student8b, hard: base70b_w4a16, router: rules_v2}
speculation: {default: ngram, controller: load_aware, max_gamma: 5}
eviction_policy: typed_v1       scheduler: critical_path
```

Appendix D — Formulas and Derived Numbers Used

Item	Formula / value	Tag
Weight memory	*Pb*/8 bytes	P1 §2.1
KV bytes/token	$2 \cdot L \cdot H_{kv} \cdot d_h \cdot b$ (Llama-3-8B-class: 32·8·128·2·2 = 131,072 B = 128 KiB ⇒ 16 GiB at 128K)	P3 F1

70B KV (80 layers, 8 KV heads, 128)	2·80·8·128·2 = 327,680 B = 320 KiB/token	P1 §19.3, derived
Bits per element	NVFP4 4 + 8/16 = 4.5; MXFP4 4 + 8/32 = 4.25; W4 g128 with 16-bit scale ≈ 4.125	P5 §3, §11.1
FP4 vs FP16/FP8 compression	16/4.5 = 3.56; 8/4.5 = 1.78	P5 §11.1
Quantiser overhead ratio	$T_q/T_g = (b_{in}+b_{out})F / (2N\beta)$: 2402/$N$ (FP4) and 937/N (FP8) at the Blackwell-Ultra vendor numbers	P5 eq. 11.1
W4A16 batch crossover (H100-class numbers)	≈ 74 (4-bit) and ≈ 295 (16-bit); speed-up exactly 4× for $B \le 74$ if dequantisation is hidden, 295/B in between	P1 §14.1.1
Speculative sampling exactness	Accept with prob. min(1, p/q); on rejection sample from $(p-q)_+/Z$; $\alpha = \Sigma \min(p,q)$; output law = p	P7 App. A.1
Low-rank break-even	$r < mn/(m+n)$; square matrices: $r < n/2$	P4 F5
Teacher cache sizes	1 B tokens × 128k vocab × 2 B = ≈ 262 TB; top-64 × 6 B = ≈ 384 GB; ratio ≈ 682×	DERIVED
Fleet KV example	200 × 32,768 × 128 KiB = 800 GiB; shared 3 GiB + 200 GiB = 203 GiB; FP8 ⇒ ≈ 101 GiB	DERIVED
Sparse-attention byte reduction	7.7× (1 indexer/layer) → 22.6× (shared across 4 layers) → 57× (one indexer for 61 layers), V3.2-style layer at 128K	P6 Ch. 3, Ch. 6

Appendix E — Glossary

Term	Meaning
AWQ / GPTQ	Activation-aware / Hessian-based post-training weight quantisation methods.
E2M1, E4M3	4-bit and 8-bit floating-point element/scale formats (exponent bits, mantissa bits).
NVFP4 / MXFP4	Block-scaled 4-bit formats: 16-element blocks with E4M3 scale plus FP32 tensor scale / 32-element blocks with power-of-two scale.
GQA / MQA / MLA	Grouped- / multi-query attention; multi-head latent attention (compressed KV).
DSA / NSA / MoBA	Trainable sparse-attention designs (indexer-based / native / block).
KL / flip rate	Distribution divergence from the parent / fraction of positions where the top-1 token differs.
Roofline / ridge	Performance bound model; the arithmetic intensity at which compute and bandwidth limits meet.
QAT / QAD	Quantisation-aware training / quantisation-aware distillation.
RHT	Random Hadamard transform (outlier-spreading rotation).
SR / RTN	Stochastic rounding / round-to-nearest.
EAGLE-3 / SSD	Feature-fusing speculative drafter / speculative speculative decoding (Saguaro).
CAS	Content-addressed store.
layout_id	Identifier of a hardware-specific weight/scale memory layout recorded in an artifact manifest.

Appendix F — Source Documents and Caveats

1. Paper 1 — *Quantization of Large Language Models* (29 Oct 2026). Subjective priority scores; some numbers labelled "approximate".
2. Paper 2 — *Pruning and Sparsity in Large Language Models* (29 Oct 2026). Figures marked † quoted as recalled; ‡ checked against abstracts.
3. Paper 3 — *KV-Cache, Attention, and Context Compression* (29 Oct 2026). Numbers tagged cited / derived / illustrative / synthetic.
4. Paper 4 — *Distillation, Low-Rank and Tensor Factorization, Parameter Sharing, and Lossless Weight Compression* (29 Oct 2026). Priority scores subjective.
5. NVFP4 paper — *NVFP4 and Microscaled 4-Bit Floating-Point Formats for LLM Training and Inference* (27 Oct 2026). Evidence tags S/V/D/E/U; no Blackwell hardware used; CPU toy-scale simulations.
6. Sparse-attention paper — *Learned Sparse and Hybrid Sparse–Linear Attention for Long-Context LLM Inference* (27 Oct 2026). Evidence ledger; vendor claims flagged.
7. Speculative-decoding paper — *Next-Generation Speculative Decoding: EAGLE-3, P-EAGLE, and SSD* (27 Oct 2026). Labels [S]/[D]/[M]/[V]; no real-GPU experiments.

LIMITS OF THIS PLAN

This plan was derived from the seven attached documents only; it does not re-verify their citations, and it does not rely on any hardware measurement of ours. Where this plan states a hardware feature by chip family (§6.5) it is a hypothesis for the capability probe to confirm. Numbers tagged TARGET are our proposals. UI mockups use placeholder data. The multi-agent track extends beyond what the papers evaluate and is explicitly gated.